EVOKE: A KV-Cache Memory Hierarchy with Recompute-Free Block Recovery for Agent Working Memory

Anish Shrestha
Independent Researcher
siranishshrestha@gmail.com
github.com/Anyesh/EVOKE

Abstract

Long-context agent sessions accumulate context (files read, tool output, search results) faster than a fixed KV budget can hold, and the usual responses are dead ends: truncate the oldest history, or re-prefill it whenever the agent refers back. EVOKE (EVict and recOver KV cache Entries) makes eviction reversible, turning the KV cache into an OS-style memory hierarchy for the agent's working set. A recompute-free splice writes the saved K and V tensors back into the active cache with a single RoPE re-anchoring rotation, at the cost of the tensor transfer; recovery is keyed by block identity, not retrieval over re-encoded text (where RAG would substitute). We add three KV-paging primitives to a llama.cpp fork (block save, block load with re-anchoring, per-layer attention capture), wrap them in a Python policy, and evaluate four model families on a single 16 GB consumer GPU. Three findings. **(1) Fidelity:** a recovered block is as usable for recall as a fresh re-decode (100% vs 100%; the evicted-not-restored floor is 0–5%) on pure-attention and hybrid models and under $\sim$128 evictions of churn, which the other findings depend on; the save+load round-trip is 5.9–7.5$\times$ faster than re-prefill on Qwen 2.5 7B (2.6–2.8$\times$ on Llama 3.1 8B). **(2) Recovery is the dividing line:** every recovery-less baseline (recency, StreamingLLM, H2O (Zhang et al., 2023), SnapKV (Li et al., 2024), reimplemented per their published configs) stays $\leq$43% on multi-fact while every recovery-bearing policy reaches $\geq$48%, holding on hybrid Mamba/Attention (68%) and MoE-with-thinking (52%); a 15-seed sweep against a same-substrate InfLLM (Xiao et al., 2024b) (K=8) shows a budget crossover (InfLLM separates at b=512, tie at b=1024, EVOKE leads at b=2048 with overlapping CIs). **(3) Scale:** EVOKE holds a 66K-token session in an 8K budget at $4.1\times$ lower peak KV where a no-eviction run hits the context wall and cannot complete. Recompute-free recall of evicted KV follows ArkVale (Chen et al., 2024); EVOKE adds identity-keyed addressing and re-anchors recovered blocks to a new live position, which Section 7.8 shows helps near the context edge.

1 Introduction

A long-running LLM agent accumulates context across a session: files it reads, tool output, search results, prior reasoning. When that working set outgrows the KV budget, the usual handling mirrors LLM serving's two options. The harness either truncates the oldest history in the spirit of StreamingLLM, or it re-prefills the conversation on every step, which is what the OpenAI chat-completions default plus prefix caching amounts to. Truncation discards information the agent may need again later. Re-prefill is expensive and pays the cost on every step regardless of whether the

Preprint. Under review.

prior context turned out to matter. Neither approach has a mechanism to bring back evicted content once the agent refers to it again.

EVOKE treats the KV cache as a memory hierarchy with reversible eviction (Figure 1). Hot tokens stay resident while cold blocks move to host RAM at metadata cost; when a future turn needs an evicted block, a recompute-free splice writes the saved tensors back into the active cache through a single RoPE rotation. Because the splice returns the bytes the model first computed, recovery is keyed by block identity rather than by retrieval over re-encoded text. Section 4 develops this distinction in detail.

The contributions of this paper are three:

1. **Three KV-paging primitives added to a forked llama.cpp.** `llama_kv_block_save` and `llama_kv_block_load` serialize a position range's K/V tensors to a host buffer and splice them back into the unified cache with per-cell RoPE re-anchoring (Section 3). `llama_attn_capture_*` taps per-head softmax attention weights from one or more transformer layers into a host buffer once per decode (Section 3.3). Together they enable recompute-free, identity-keyed K/V recovery and attention-aware eviction scoring. The fork extends to hybrid Mamba+Attention memory and mrope position shifts, so the same primitives work across pure attention, hybrid Mamba/Attention with mrope, and MoE with thinking, all on a single 16 GB consumer GPU (Section 6).
2. **A Python policy layer and OpenAI-compatible server** that turn the primitives into a usable system. The eviction scorer combines retrieval-tuned embedding coherence (`bge-small-en-v1.5`), the model's own attention mass, recency, and harness-supplied priority. Recovery is routed through three pluggable backends (discard, breadcrumb, kv_restore). A 2^3 smart-recovery factorial attributes most of the multi-fact gain to a single policy decision: scoring blocks against the raw user-message text through a dedicated retrieval embedder rather than pooling LM hidden states lifts pass rate from 14 % to 86 % (+72 pp, Appendix A.4); running the recovery splice before the new user-message tail is decoded contributes a conditional +20 pp on top, and the resident-gate has no measurable effect on the benchmark we ran the factorial against. The server reuses prefix-matched KV across turns and pools per-session KV state; in our measurements, a single Qwen 2.5 7B instance sustains 16 concurrent sessions with all 80 of 80 recall probes passing and a p99-to-p999 tail spread under 0.13 s (Section 4, Section 5, Appendix A.10).
3. **A head-to-head against seven baselines across four model families.** The recovery-less baselines (recency, StreamingLLM, H2O (Zhang et al., 2023), SnapKV (Li et al., 2024), plus EVOKE's own no-recovery backends) are reimplemented per their published configurations on the EVOKE harness, so the recovery-vs-no-recovery comparison isolates scheduling from engine differences. Recovery-bearing policies separate from recovery-less ones on the multi-fact partition, and the partition holds at $b{=}1024$ across the hybrid Mamba/Attention and MoE-with-thinking variants. Within the recovery-bearing cluster, a 15-seed multi-fact sweep against a same-substrate InfLLM (Xiao et al., 2024b) adaptation at $K{=}8$ shows a budget-axis crossover. Both schedulers ride the same recovery splice, which carries the contribution; K and scheduler choice are tuned by budget. The `kv_block_load` primitive runs in 0.5 to 16 ms across block sizes from 20 to 1280 tokens; partition pass rates by benchmark and architecture, the speedup bands, NIAH per-architecture detail, the InfLLM crossover with Wilson CIs, and the 14-turn `no_eviction`-parity head-to-head are in Section 7.

2 Related Work

Table 1 places EVOKE among KV-cache eviction and recovery methods; the paragraphs below detail each family.

Eviction-only KV compression. StreamingLLM (Xiao et al., 2024a) keeps a small set of attention-sink tokens at the cache head plus a sliding recent window and drops everything in between, working well for recency-only tasks but failing on earlier recall. **H2O** (Zhang et al., 2023) and **Scissorhands** (Liu et al., 2023) score KV entries by accumulated attention weight and drop the lowest-scoring; **SnapKV** (Li et al., 2024) and **Ada-KV** (Feng et al., 2024) subsequently critiqued the H2O family for biasing against recent tokens (cumulative scoring under-counts tail positions)

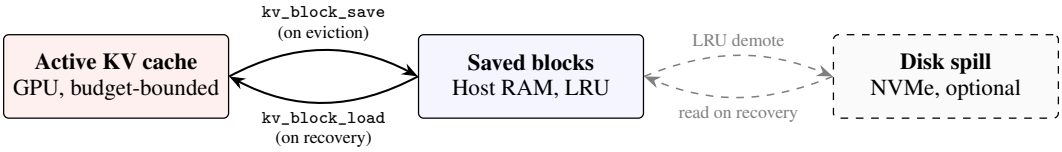

Figure 1: EVOKE's memory hierarchy. The active KV cache holds the hot working set on GPU within a configured budget. On eviction, a block's K and V tensors are serialized to host RAM via `kv_block_save`; recovery reverses the operation with `kv_block_load`, splicing the original tensors back into the active cache at a new logical position with one RoPE phase shift and no forward pass. An optional disk spill tier extends host RAM at NVMe latency.

Method	Recalls evicted KV	Recall keyed by	Recall placement	Substrate
Recency / StreamingLLM	no	—	—	attention
H2O / SnapKV / Scissorhands	no	—	—	attention
Compressive transformers	no (compress)	—	—	attention
RAG	re-encodes text	similarity	re-encoded	external
InfLLM	yes (no recompute)	similarity	unified splice	attention
ArkVale	yes (no recompute)	importance digest	original position	attention, per-layer
EVOKE	yes (no recompute)	identity / similarity	re-anchor / sparse	attention, hybrid, MoE

Table 1: Where EVOKE sits among KV-cache eviction/recovery methods. Recovery-less methods (top) cannot bring back an evicted block once it is dropped. Among recovery-bearing methods, EVOKE recalls by block identity into the unified contiguous cache (re-anchoring the recovered block to a new position) and runs across attention, hybrid Mamba/Attention, and MoE substrates. ArkVale recalls by a per-layer importance digest into a per-layer paged cache; Section 7.8 and Section 8 discuss the placement and per-layer differences.

and proposed single-pass selection and head-aware budgets. All treat eviction as one-way: dropped K/V is gone. EVOKE adopts the same attention-weight signal in its multi-signal scorer (Section 4), keeps the sink-protection idea (sink-tagged blocks score 1.0 unconditionally), combines attention with recency as separate weighted signals to sidestep the recent-bias issue, and pairs eviction with a recovery path so an eviction that turns out to be wrong is recoverable rather than terminal. H2O and SnapKV are reimplemented as same-substrate baselines atop EVOKE's `AttentionScorer` infrastructure (H2O: cumulative attention with no decay, 10% recent-tail guard; SnapKV: one-shot 32-token observation-window snapshot at the end of every user-message decode) and benchmarked head-to-head in Sections 7.3 to 7.4.

Retrieval and external-memory families. RAG (Lewis et al., 2020) stores text in a vector database and re-encodes retrieved passages on every call; the model has no continuity with how those tokens were originally attended. EVOKE keeps the original K and V tensors the model computed at first sight of the tokens and splices them back at their original positions, with Section 4 drawing the precise line between identity-addressed recovery (which may still use similarity to choose *which* blocks to bring back) and similarity-retrieved re-encoding. **InfLLM** (Xiao et al., 2024b) maintains a block-level external KV memory and selects blocks back into the attention window via similarity, holding retrieved blocks in a separate memory segment the attention layer reads alongside the active window. EVOKE shares the block-level memory-hierarchy framing but splices recovered blocks back into the unified contiguous KV cache (RoPE re-anchored to the new logical position), and benchmarks InfLLM as a same-substrate row in Sections 7.3 to 7.4: aggressive eviction to a sinks-plus-local-window footprint with per-turn top-K=8 smart recovery splicing the externally-held blocks back into the unified cache. **Compressive transformers** (Rae et al., 2019) and related infinite-memory architectures compress old KV into a smaller summary; EVOKE does not compress, trading host RAM proportional to evicted content for exact recovery. **ArkVale** (Chen et al., 2024) is the closest prior work on the recall axis: it backs up evicted KV pages to host memory and recalls them into a budget-sized dense cache on demand. It recalls each page at its original absolute position, where EVOKE re-anchors a recovered block to a new contiguous tail position; Section 7.8 measures when that re-anchoring helps and when it hurts. **CacheFocus** (Lee et al., 2025) also recomputes RoPE to re-position cached keys, but over independently-prefilled RAG chunks rather than blocks from one

live sequence. **MemGPT** (Packer et al., 2023) frames agent context as OS-style paged memory; it pages text in and out of the prompt, where EVOKE pages the K and V tensors themselves.

KV save-restore systems. A separate line of work treats the KV cache as a storage tier rather than a working set. **CachedAttention** (Gao et al., 2024) maintains a hierarchical KV store across HBM, host memory, and disks (named AttentionStore), saving a session's cache when the conversation goes inactive and reloading it on the next turn with layer-wise pre-loading, asynchronous saving, and positional-encoding decoupling so the saved cache survives context-window truncation. **CacheGen** (Liu et al., 2024b) compresses KV into compact bitstreams for cross-node network streaming with bandwidth-adaptive per-layer compression. **CacheBlend** (Yao et al., 2025) targets cross-request RAG: it reuses precomputed per-chunk KV at non-prefix positions and selectively recomputes K and V on a small subset of tokens (under 15%) to restore the cross-attention that prefix-only reuse cannot recover. LMCache (cited below) integrates CacheBlend as its non-prefix reuse path on top of vLLM and SGLang. EVOKE differs in granularity and timing: blocks are spliced back into a single agent session's active cache mid-generation, at sub-turn block granularity, driven by an in-session similarity selector. Recovery is identity-keyed (the exact saved K and V bytes are written back, never re-encoded or blended), distinguishing EVOKE from CacheBlend's blended recompute and CacheGen's lossy compression-decompression cycle. The closest neighbour on the identity-keyed axis is CachedAttention, which reloads a whole inactive session at turn boundaries; EVOKE re-attaches one block at a time inside a live decode loop driven by in-session selection signals.

Cache infrastructure and mechanics. vLLM / PagedAttention (Kwon et al., 2023) splits the KV cache into fixed-size physical blocks indexed by a virtual page table; **SGLang's RadixAttention** (Zheng et al., 2024) extends this with a tree of cached prefixes shared across requests. **LMCache** (LMCache Team, 2024) stores prefix-matched KV bytes in a layered cache shareable across vLLM instances, and **DistAttention** (Lin et al., 2024) disaggregates attention computation across a cluster. These cache infrastructures and EVOKE's policy layer address orthogonal concerns: paged virtual addressing handles how cached bytes move and are shared across requests, while EVOKE handles which blocks to evict and how to splice them back. EVOKE's recovery primitive is a typed read/write of K and V keyed by (layer, head, position range), so its algorithmic content is independent of whether the underlying KV layout is contiguous (as in our llama.cpp prototype) or virtually paged. Porting the C primitives onto a page-table-resolved gather/scatter is engineering work we name explicitly as future work (Section 9); this paper presents the primitive and policy on a flat unified cache as the simplest substrate that exposes both K and V tensors at addressable offsets. **RoPE re-anchoring** is enabled by RoFormer (Su et al., 2024): shifting a K row from one position to another reduces to a single rotation, machinery llama.cpp already exposes for its own `seq_add` shift, which EVOKE reuses on the recovery path. **Retrieval embeddings** for smart-recovery scoring use BGE (Xiao et al., 2024c), specifically `bge-small-en-v1.5` via the `fastembed` runtime; the choice of a dedicated retrieval embedder rather than LM hidden states makes block-vs-query similarity discriminative on retrieval-style workloads (Section 7.3).

3 The C Primitives

Two primitives, implemented in `src/llama-context.cpp` and exposed in `include/llama.h` (signatures verbatim in Appendix C), save a range $[p_0, p_1)$ of cells from sequence `seq_id` into a host buffer and restore it at a new start `new_p0`. Restoration must produce the same K and V tensors the model would attend to if positions `new_p0` through `new_p0` $+ (p_1 - p_0)$ had been decoded in place.

`llama_kv_block_save` reuses the existing `state_write_*` paths filtered to cells whose `pos` is in $[p_0, p_1)$, reading each cell's K and V row plus its original `pos` (the deferred RoPE shift needs it). `llama_kv_block_load` reuses the `dest_seq_id != -1` read path but clears only $[$`new_p0`$,$ `new_p0` $+$ `n_cells`$)$ rather than the whole sequence, writes the saved K and V into a fresh ubatch slot, and re-anchors RoPE by setting `shift`$[i] =$ `new_pos` $-$ `original_pos` per cell before running the existing K-shift graph. V is never rotated.

This path runs no model forward pass: `llama_decode` is never called, and the per-block cost is $O($`n_cells` $\times$ `n_layers` $\times$ `n_embd_kv` $\times$ `sizeof(dtype)`$)$ of host-to-device memcpy plus the single K-rotation graph llama.cpp already runs for its own `seq_add` path. Throughout this paper,

"recompute-free" is shorthand for "no model forward pass," not literal zero compute; the latency numbers in Section 7.1 measure end-to-end load including the rotation.

3.1 Flash Attention is non-optional

With Flash Attention (Dao et al., 2022; Dao, 2023) disabled, V is stored transposed (`v_trans=true`) and the save/load V loop iterates `n_embd_v_gqa` times per layer, which for Qwen 2.5 7B is 14,336 synchronous CUDA launches per load. With Flash Attention on (`v_trans=false`), V is row-aligned like K and the fast path runs 56 launches per load (one K and one V per layer). For a 20-token block on the eval host, `kv_block_load` drops from 133 ms (FA off) to 0.48 ms (FA on). Flash Attention is therefore a performance precondition: the splice is functionally correct with FA off, but the $\sim$280$\times$ slowdown collapses the speedup the primitive exists to deliver.

Deployment-substrate consequence. The requirement excludes substrates where FA is unavailable or off by default: CPU-only inference, the older Vulkan and Metal backends without the fused softmax path, and pre-Ampere CUDA. EVOKE's usable surface is Ampere-or-later CUDA with FA on, the same surface modern paged-attention stacks (vLLM, SGLang) target by default, so it is consistent with production assumptions but a real exclusion that substrate ports must reckon with.

3.2 Cross-architecture extensions

The extension point is the fork helper `llama_get_kv_cache`, which resolves a context's `llama_memory_t` to its attention KV cache; we extend it to unwrap `llama_memory_hybrid` and `llama_memory_hybrid_iswa` so the same save/load/re-anchor primitives reach the attention component of hybrid and iSWA models (commit detail in Appendix C). Hybrid models carry a fixed-size recurrent state per layer that does not need eviction, so EVOKE operates on the attention component alone and the recurrent state is carried forward untouched.

For mrope models (Qwen 3.5 and later, `n_pos_per_embd > 1`) the upstream `seq_add()` and `get_can_shift()` guards return false; we remove them, and `build_rope_shift`'s existing mrope workaround treats the temporal shift as a whole-vector rotation, which is semantically correct for our position-only re-anchoring of the temporal `pos`.

For hybrid Mamba plus Attention memories, `seq_rm` dispatches to the recurrent half, which refuses partial tail rollback (a Mamba state cannot be sliced) and returns false without mutating either sub-cache. A caller that reads the false as success and advances its bookkeeping corrupts the cache on the next decode; EVOKE's `evict_ranges` reads the return value and leaves bookkeeping intact on rejection. Two attention-only primitives (`llama_kv_block_seq_rm`, `_seq_add`) bypass the hybrid wrapper for a future no-shift eviction mode the current policy does not yet exercise. For iSWA models (Gemma 3 and 4), save/load cover both base and SWA sub-caches via `llama_get_kv_cache_swa()`; the fork primitives are verified, but the Python wiring to recover a Gemma model end-to-end is not yet in place.

3.3 Attention-weight capture

The multi-signal scorer (Section 4) needs the model's own attention as a relevance signal: the bilinear form $\texttt{softmax}(QK^\top/\sqrt{d})$ that drives the forward pass. Flash Attention fuses the softmax in its CUDA kernel and never materializes the weights, and EVOKE requires FA on (Section 3.1), so reading `kq_soft_max` from the graph is not an option, and editing the FA kernel to emit weights would be a correctness-sensitive change across four backends.

The lower-risk path computes a second softmax in parallel for chosen layers. After Q and K are permuted (the same tensors FA consumes), we add a side graph node computing $\texttt{softmax}(QK^\top, \texttt{scale} = \texttt{kq\_scale}, \texttt{mask} = \texttt{kq\_mask}, \texttt{sinks})$, force-expanded so the scheduler does not drop it, and copy the result to a caller-owned host buffer when `llama_decode` returns. The main path is unchanged: FA still runs, output tokens are identical with capture off versus on, and the side-softmax is numerically close but not bit-identical to FA's internal softmax (verified below). The C API (`set_layer`, `set_layers`, `set_buffer`, `get_dims`, `get_written`) is verbatim in Appendix C.4; `set_layers` writes up to 16 layer slabs per decode at layout `[n_layers, n_query, n_heads, n_kv]`, roughly 1.8 MB per layer slab per step on Qwen 2.5 7B (four layers fit in 7.2 MB).

Validation. `scripts/verify_attn_capture.py` decodes a 31-token prompt on Qwen 2.5 7B with layer 20 tapped and asserts per-query softmax sums to 1.0, values in $[0, 1]$, the causal mask respected (above-diagonal weights exactly zero), and legal nonzero cells matching $\sum_i (i+1) \cdot n_{\text{heads}}$ exactly. `scripts/verify_attn_capture_multi.py` captures layers 4, 14, 20 in one 29-token decode and asserts the same invariants plus distinct per-layer patterns. Both pass on the eval host; the side-compute matches non-FA mode below the f32 rounding tolerance. The scorer experiments in Appendices A.1 and A.6 use a single mid-layer (layer 20 for Qwen 2.5 7B), sufficient to drive the signal; multi-layer aggregation is exposed for future scorers.

3.4 Why eviction does not corrupt the cache

Eviction is two engine calls. `seq_rm(seq, p0, p1)` frees the cells in $[p_0, p_1)$, releasing their K and V rows; no dangling reference survives because Q is never cached (it is recomputed each decode step). `seq_add(seq, p1, end, delta)` with $\delta = -(p_1 - p_0)$ updates the survivors' position metadata to close the gap and queues a deferred RoPE shift by δ on their K rows, moving no bytes. Because RoPE expresses position as a multiplicative phase with $K(\text{pos}) \cdot Q(\text{pos}_q)$ collapsing to a function of $(\text{pos} - \text{pos}_q)$ (Su et al., 2024), re-anchoring a K row from p to $p + \delta$ is exactly one rotation $R(\delta \cdot \theta_i)$ per dimension pair, so after the lazily-applied shift $Q_{\text{new}} \cdot K_{\text{survivor}}$ returns the same relative-position dot product as if the evicted range had never been decoded; V is positional-free and untouched. The mrope case follows via the temporal axis (Section 3.2); the hybrid case forwards `seq_rm`'s rejection of partial recurrent rollback without mutating the cache. Eviction is therefore a topological cut, not partial erasure: its failure mode is information loss (the model lacks K and V for the evicted fact and hallucinates or refuses), not corruption, which is what `kv_restore` addresses by splicing the saved K and V back at a fresh contiguous position with one further RoPE shift.

4 The EVOKE Policy Layer

The policy layer lives in `evoke/manager.py`, `evoke/scorer.py`, and `evoke/attention_scorer.py`. Each active block carries a score in $[0, 1]$ used by the watermark policy: when `active_tokens` crosses `high_watermark` $\cdot$ `budget`, the manager evicts the lowest-scoring blocks down to `low_watermark` $\cdot$ `budget`.

Which decision drives the gain. We describe the full multi-signal scorer below for completeness, but the 2^3 factorial in Appendix A.4 attributes the bulk of the smart-recovery gain on NIAH at b=512 to one decision (K1: scoring blocks against the raw user-message text through a dedicated retrieval-tuned embedder rather than pooling LM hidden states, marginal +72 pp), with a conditional +20 pp from running the splice before the new tail decodes (K2, only when K1 is on) and no measurable effect from the resident-gate (K3). The scorer below is therefore the framework within which the core primitive (Section 3) and the retrieval embedder compose, not the contribution itself. The attention-mass signal is an improvement that pays off when attention concentrates on a single recoverable target (Appendix A.1); on multi-fact its value is budget-dependent and a larger-K pure-retrieval recovery can outperform it (Section 7.4).

The score is a weighted linear combination of up to five signals, lifted by a source-type floor, multiplied by a harness-supplied priority, and clamped:

$$\textit{raw} = \frac{w_{\textit{attn}} \cdot \textit{attn} \;+\; w_{\textit{rec}} \cdot \textit{recency} \;+\; w_{\textit{coh}} \cdot \textit{coherence} \;+\; w_{\textit{recov}} \cdot \textit{recovery\_strength}}{\sum w_i}$$

$$\textit{score} = \min\bigl(1,\ \max(\textit{raw},\ \textit{source\_floor}) \ \cdot\ \textit{block.priority}\bigr)$$

Sink blocks short-circuit to *score* $= 1.0$ unconditionally. The four signals in the numerator are explained below; the fifth (*recovery_strength*) is a model-history term covered separately in the recovery-aware paragraph.

Attention (Section 3.3). Per-block softmax mass landing in that block over the last `n_window` decode steps, EWMA-weighted by `decay`. Defaults are a 64-step window and a decay of 0.95. This is the model's own vote: blocks the recent query tokens actually attended to score high. When attention capture is unavailable (a stock llama.cpp build, or `w_attention=0`), this signal is absent and the score falls back to recency plus coherence.

Task-focus coherence. The scorer keeps a single task focus embedding rather than averaging the last N messages. On each new user message the focus updates via EMA when the message is coherent (cosine to the focus $\geq$ `task_boundary_threshold`, default 0.3 in the cosine sense, not a weight) or snaps to the new message, detected implicitly via the cosine drop or signaled by `evoke_task_boundary=true`. Snapping drops blocks coherent with a prior task in one scoring pass instead of decaying over five turns under a rolling average.

Per-block embeddings come from two sources, switchable via `EvokeConfig.use_retrieval_embeddings`. The default pools the model's own hidden states across block tokens (cheap, already computed), but the LM hidden state's strong common-mode component collapses cosine similarities into a narrow 0.85 to 0.93 band across any two paragraphs in a corpus, leaving no retrieval headroom. The opt-in retrieval mode embeds each block's decoded text through `BAAI/bge-small-en-v1.5` (`fastembed`, $\sim$30 MB, $\sim$10 ms per text on CPU), widening the band to 0.4 to 0.9 so the smart-recovery policy (Section 5) can separate a needle block from haystack noise. Block and probe embeddings must share a space; the policy routes both through the same embedder.

Recency. Exponential in the per-block distance from the current write position. A stability prior that prevents thrashing on a single high-attention spike.

Sink protection and source-type floors. Blocks marked `is_sink=True` (the first `sink_count` positions) score 1.0 unconditionally, which is the StreamingLLM-style attention anchor. USER and ASSISTANT turns get a score floor (defaults of 0.6 and 0.5 respectively) so that the conversation backbone is not evicted before document content.

Recovery-aware eviction. A fifth, model-history-driven signal records that the model recently signaled a block matters by recovering it. Each `ActiveBlock` carries a `recovery_strength` (default 0.0); `recover()` sets it to `recovery_strength_init` (default 1.0), and `tick_turn()` multiplies every block's strength by `recovery_decay` (default 0.7) each user turn. The scorer adds $w_{\text{recovery}} \cdot$ `recovery_strength` to the sum. Default `w_recovery` = 0 preserves the four-signal behavior; setting it positive closes the recover-then-immediately-evict thrash the session-length sweep (Appendix A.7) exposed at $T{=}28{+}$: a freshly-recovered block survives the next eviction pass even when recency and coherence would re-evict it, then decays back to ordinary eligibility over $\sim$5 turns. The mechanism is structural: eviction respects the model's own prior relevance signal rather than recency and coherence alone.

EVOKE ships two scorer recipes. The `EvokeConfig` default (`w_attention=0.0`, `w_recency=0.4`, `w_coherence=0.6`) is the fork-independent fallback that runs unmodified on a stock llama.cpp build lacking attention capture. The recommended config (`w_attention=0.5`, `w_recency=0.2`, `w_coherence=0.3`) opts in to the multi-signal scorer when the fork build is available. Across every head-to-head in Section 7 the two recipes are statistically identical: NIAH on Qwen 2.5 7B is 96/100/100% for both at budgets 512/1024/2048 (Section 7.3), NIAH on Llama 3.1 8B is 88% across all capture-layer choices for both (Appendix A.9), and multi-fact $n{=}15$ CIs overlap at every budget (Section 7.4). We recommend the multi-signal recipe not because it wins but because attention is a more principled relevance signal (the model's own per-block pattern, not a heuristic prior); the default exists for substrate-constrained deployments. The `w_coherence` weight (0.3 or 0.6) and `task_boundary_threshold` (0.3 cosine) are unrelated despite the shared number: weights are linear-combination coefficients, the threshold is a topic-shift cosine cutoff.

Harness-supplied tags. The chat-completions request gains three fields: `evoke_priority` (a non-negative float multiplying the final score), `evoke_pinned` (excludes the block from eviction entirely), and `evoke_task_boundary` (forces the coherence focus to snap). A harness like opencode or Claude Code can set `priority=2.0` on central file reads to keep them resident and `priority=0.3` on one-shot tool output so it drops first under pressure. The fields default to `1.0`, `false`, `false`, so harnesses that ignore them get the model-and-heuristic behavior. Pinned blocks are excluded from `_evictable_blocks` alongside sinks, and `pin_generated=True` (default) protects blocks created during the decoding turn so generation does not evict itself.

Recovery backends (`evoke/recovery.py`). Three pluggable strategies share a `RecoveryBackend` protocol.

- `DiscardBackend`: evicted content is gone. The agent must re-derive it.
- `BreadcrumbBackend`: a compact marker (`key`, token count, eviction step) is retained per evicted block. The agent (or harness) can choose to re-fetch the original text and call `add_context` to re-decode it.
- `KVRestoreBackend`: per-block K and V are saved to host RAM via `kv_block_save`. `recover(key)` looks up the saved buffer, calls `kv_block_load` at the next available position, and re-anchors RoPE. When a configurable RAM budget would otherwise drop an LRU victim, the backend can optionally spill the K and V bytes to disk under `kv_restore_spill_path`. Recovery then reads back from disk before splicing, at the cost of one NVMe read.

Block identity and where EVOKE sits relative to RAG. EVOKE is a hybrid by construction: a retrieval-shaped selection layer chooses which evicted block to bring back, and an identity-keyed substitution layer splices that block's original K and V into the cache. Selection looks like retrieval (smart top-K scoring in Section 5 embeds blocks and queries with `bge-small` and picks by cosine), but substitution does not: every block carries a stable `key` such as `"file:config.py#0"` or `"user#42"`, and the recovery call names that key and pastes back the exact bytes the model computed at first decode, never a re-encoding or similarity-retrieved alternative. The systems content lives at the substitution layer (Section 3); the selection layer is acknowledged retrieval, intentionally so.

5 The OpenAI-Compatible Server

The chat-completions API is stateless: every request resends the full history, and the canonical response is to re-prefill from scratch or, with prefix caching, reuse the unchanged prefix. EVOKE's server (`evoke/server.py`, `evoke/session.py`) makes the session the unit instead. A persistent `Session` with one `EvokeManager` outlives every request: incoming messages are templated and tokenized, prefix-matched against the session's `cached_tokens` to find the longest prefix already resident, and the tail past that point is decoded through `add_context_tokens`, which creates blocks and fires watermark eviction (Section 4). The SSE stream runs a three-state machine (thinking, tool-locked, normal) that strips `<think>` traces and parses `<tool_call>` XML into OpenAI's `tool_calls` shape. A `SessionPool` lets one model in VRAM host many concurrent sessions, each with its own KV identity: it holds per-session Python state plus a serialized engine snapshot, and on an `X-EVOKE-Session` switch serializes the outgoing KV state and restores the incoming snapshot in one memcpy proportional to active KV size with no recompute, evicting the least-recently-touched inactive session past `max_sessions` (default eight). `scripts/verify_multi_session.py` confirms two interleaved sessions retrieve their own planted facts without cross-contamination.

Smart recovery. The session runs smart top-k recovery on each user turn (default $k = 4$, cost capped at k `kv_block_load` calls). A 2^3 factorial over three policy choices (Appendix A.4) attributes the bulk of the NIAH improvement to one: scoring blocks against the raw user-message text through a dedicated retrieval-tuned embedder rather than an LM-hidden-state average. Its effect at NIAH b=512 on Qwen 2.5 7B is +72 pp (14 % $\rightarrow$ 86 %) because the retrieval path widens the cosine band enough to separate the needle from haystack noise. Running recovery *before* the new user-message tail is decoded interacts with that choice: with the retrieval embedder it adds +20 pp (76 % $\rightarrow$ 96 %) as recovered blocks land as earlier context the freshest probe attends back to, but with the LM-hidden-state embedder it hurts by -28 pp by landing wrong blocks in the freshest slot. A resident-gate excluding the current-turn user message has no measurable effect at b=512 in the factorial; it guards a depth-95 % failure mode (Section 7.3) at depths the factorial does not sweep.

6 Models Supported

End-to-end verified on a single GPU host (RTX 4070 Ti SUPER, 16 GB VRAM, CUDA 13.1):

Model	Architecture	Status
Qwen 2.5 7B Instruct	Pure attention, standard RoPE	full bench suite (NIAH, multifact, agent_bench, baseline_bench, concurrency); NIAH 96 to 100% across budgets 512/1024/2048
Llama 3.1 8B Instruct	Pure attention, standard RoPE, 32 transformer layers	full bench suite (Appendix A.9): NIAH 76 / 88 / 88 % across budgets 512/1024/2048; multifact budget crossover reproduces; agent_bench PASS at every budget; `kv_block_load` 1.78 to 15.66 ms across block sizes 20 to 1280 tokens (Section 7.1)
Qwen 3.5 9B	Hybrid Mamba/Attention, mrope, thinking	full NIAH grid (25 cells, 84% pass; 4 fail cells are generation-side truncation of the recovered value, not primitive misses) and full multifact grid (25 cells, n=5 seeds, 68 % [48.41, 82.80] vs 0 to 8% recovery-less, H2O 8%) at budget 1024; thinking-strip selectable via `EVOKE_SUPPRESS_THINKING_STRIP`. Budget sweep (512/2048) not run due to thinking-mode generation cost.
Qwen 3.6 35B-A3B (IQ2_M)	MoE attention, mrope, thinking	full NIAH grid at budget 1024 (64% vs baselines 16%) and full multifact grid (25 cells, n=5, 52 % [33.50, 69.97] vs 0% every baseline including H2O); aggressive IQ2 quant likely accounts for the lower absolute number. Budget sweep not run; 35B at IQ2_M sits near the 16 GB VRAM ceiling at model weights alone.

7 Evaluation

The evaluation builds from the mechanism up to the system. Section 7.1 measures the recompute-free primitive (Section 3) against re-prefilling the same tokens; Section 7.2 tests whether a recovered block is as usable for recall as a fresh re-decode. Sections 7.3 and 7.4 measure the policy layer (Sections 4 and 5) at recall tasks, separating the structural recovery-bearing-vs-recovery-less divide from the narrower InfLLM head-to-head. Section 7.5 shows the value at the motivating scale (holding a session past the context window); Section 7.6 measures wall-clock against a vanilla server and truncation, where we make no speed claim; Section 7.7 reports the negative substrate-portability result. Detailed ablations (scorer-knob factorial, host-RAM degradation, session-length scaling, concurrency, KV-quant) are in Appendix A.

7.1 Primitive latency: recompute-free recovery versus re-prefill

Engine-level profile on the eval host (RTX 4070 Ti SUPER, 16 GB VRAM, Flash Attention on), generated by `scripts/profile_recover.py` across block sizes 20 to 1280 tokens. `kv_block_load` is the splice-and-rotate cost; `re-prefill` is the equivalent `process_tokens` call, the alternative if the same tokens had to be re-encoded (Figure 2, Table 2).

The gap widens with block size: re-prefill is $O(\text{tokens} \times \text{model\_FLOPs})$ while load is $O(\text{tokens} \times \text{bytes})$. At 1280 tokens host-to-GPU transfer dominates and `kv_block_load` is 32 times faster than re-prefill on Qwen. Re-prefill is nearly identical between architectures (a 7B-class forward pass dominated by model FLOPs); the save and load curves are scaled up by Llama's larger per-token K/V footprint ($\sim$131 KB/token vs Qwen's $\sim$60), dropping the speedup band from 20 to 32$\times$ on Qwen to 7 to 15$\times$ on Llama. Recompute-free recovery remains strictly faster than re-prefill at every block size on both architectures.

Full-lifecycle cost. `save` is paid at eviction time and `load` at recovery time. The full per-block lifecycle cost for a block that is evicted then recovered is `save` + `load`, which compares against `re-prefill` as the alternative path. On Qwen that is 1.58 ms vs 11.90 ms at 20 tokens (7.5$\times$)

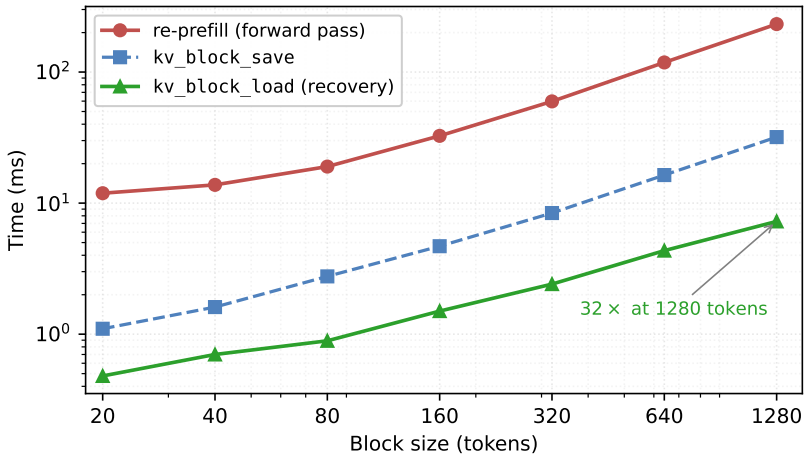

Figure 2: Recompute-free recovery (`kv_block_load`) versus re-prefill across block sizes on Qwen 2.5 7B (RTX 4070 Ti SUPER, Flash Attention enabled). Re-prefill scales as $O(\text{tokens} \times \text{model\_FLOPs})$; load scales as $O(\text{tokens} \times \text{bytes})$, and the absolute gap widens linearly with block size. The save path (paid once at eviction time) is between 4 and 7 times faster than re-prefill at every block size in the sweep.

	Qwen 2.5 7B Q4_K_M				Llama 3.1 8B Q4_K_M			
n_tok	save	load	re-prefill	speedup	save	load	re-prefill	speedup
20	1.10	0.48	11.90	25×	2.65	1.78	12.58	7.1×
40	1.61	0.70	13.78	20×	3.82	1.94	14.62	7.5×
80	2.76	0.89	19.00	21×	5.82	2.46	19.94	8.1×
160	4.69	1.50	32.60	22×	10.30	3.60	32.55	9.0×
320	8.40	2.41	59.72	25×	19.82	5.77	61.23	10.6×
640	16.37	4.34	118.36	27×	39.05	8.87	121.83	13.7×
1280	31.90	7.25	232.18	32×	74.35	15.66	236.89	15.1×

Table 2: `kv_block_save`, `kv_block_load`, and equivalent `re-prefill` times in milliseconds across block sizes 20 to 1280 tokens. Speedup is re-prefill divided by load (the latency a deployer pays on recovery). The save path is paid once at eviction.

and 39.15 ms vs 232.18 ms at 1280 tokens (5.9×). On Llama the lifecycle ratio is 2.6 to 2.8×. A block evicted but never recovered pays the save cost without benefit; Section 7.6 measures the full end-to-end cost including save plus the bge-small smart-recovery selection overhead at the session level.

7.2 Recovery fidelity: recovered KV is as usable as a fresh re-decode

Speed is moot if a spliced-back block is not *usable*. We test this directly with a controlled greedy probe (`scripts/recovery_usability.py`, `scripts/churn_fidelity.py`): plant an unguessable value, evict the block, restore it, then ask for the value, scoring recall as whether the answer contains the value. Figure 3 reports it on a pure-attention model (Qwen 3-14B) and a hybrid Mamba/Attention model (Qwen 3.5 9B), both with a single eviction and under heavy churn (a target shifted by ∼128 compact-eviction position updates before it is itself evicted and restored).

Recovered recall equals re-decode recall (both 100 %) on both architectures, with and without churn, while the discard floor sits at 0–5 %. A separate byte-level check (`scripts/verify_kv_fidelity.py`) confirms the in-place restore reproduces a never-disturbed reference token-for-token (48/48 greedy continuation match). Recovery is therefore as usable as paying to re-decode, at the recompute-free cost of Section 7.1. The 16-question agentic loop in Section 7.5 shows a residual recovered-vs-re-decode gap that is a re-presentation effect of that noisy

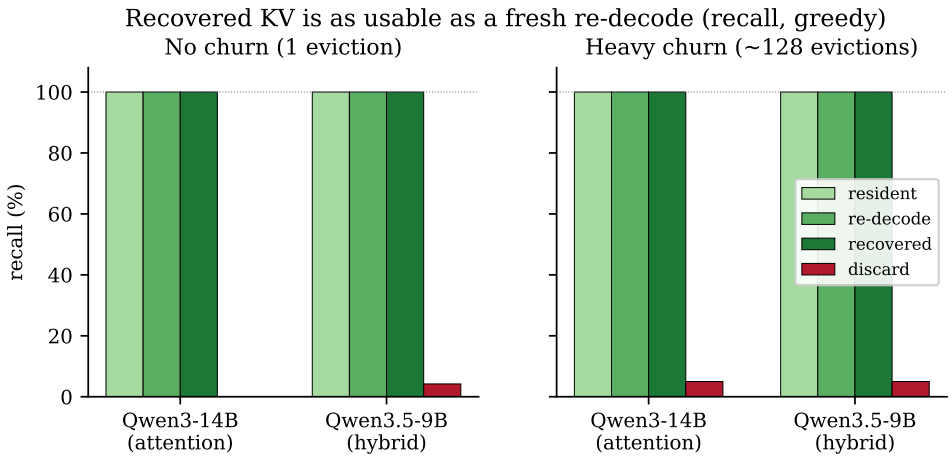

Figure 3: Recall from a recovered block matches a fresh re-decode and a never-evicted resident copy (greedy, value-in-answer), with a single eviction (left) and under ∼128 evictions of churn (right), on a pure-attention and a hybrid model. The `discard` floor (evicted, not restored) at 0–5 % confirms the probe is sensitive: the value is unrecoverable without the saved KV. That recovered ≈ re-decode *under churn* establishes that the recompute-free splice returns usable context, not merely attendable bytes.

multi-turn setting (re-decoding literally re-shows the answer text), not a fidelity loss; the controlled probe here isolates the substrate and finds no degradation.

7.3 Recovery is the dividing line, not the eviction scorer

The structural finding across the recall benchmarks is that the presence of a recovery primitive separates passing policies from failing ones, regardless of how smart the eviction selector is. We show this on three benchmarks of increasing difficulty. The divide is large because recovery-less baselines structurally cannot retrieve evicted content, so the gap quantifies what the primitive enables rather than asserting the policy out-selects H2O or SnapKV at eviction; the narrower head-to-head against the one recovery-bearing baseline (InfLLM at $K{=}8$), where the eviction scorer is on trial, is in Section 7.4.

Agent-bench: a planted fact past recency. `scripts/agent_bench.py` plants a fact in an early "config file" read (`config.py: maximum retry limit 17 attempts`), fills the working set with ten unrelated file reads, then probes the fact. By the time the probe runs, the config block has almost certainly been evicted under budget pressure. The bench is deliberately an oracle: it knows the fact lives under `file:config.py` and, for the recovery-bearing strategies, recovers exactly that key before the probe. This isolates the cost of the recovery primitive from the cost of *selecting* which block to recover.

Nine strategies at three KV budgets on Qwen 2.5 7B, `block_size=64`: five EVOKE-substrate policies (recency, streaming_llm, evoke_discard, evoke_breadcrumb, evoke_kv_restore, plus evoke_attention added in Appendix A.1) and three same-substrate baselines (h2o, snapkv, infllm). Table 3 summarizes pass status across all 27 cells; full per-cell numbers (eviction counts, recovery latency) are in Appendix A.1.

The five failing policies produce concrete refusal modes: "The maximum retry limit set in `config.py` is not explicitly mentioned" or "I cannot answer this question without inspecting the `config.py` file." The H2O cumulative-attention selector and SnapKV's question-window snapshot do not change the outcome: their eviction counts are accurate (34, 24, and 4 for H2O/SnapKV; the block holding the fact is gone), but the model has no way to recover the lost context. Even `streaming_llm`, which pins the first four blocks as sinks, fails because the planted fact sits well past the sink prefix. The four recovery-bearing policies all pass at every budget; the cost structure separates between `breadcrumb` (15 to 17 ms per recovery, re-decode) and `kv_restore` (3 to 4 ms, splice). `evoke_attention`

strategy	b**=512**	b**=1024**	b**=2048**
recency	fail	fail	fail
streaming_llm	fail	fail	fail
evoke_discard	fail	fail	fail
h2o	fail	fail	fail
snapkv	fail	fail	fail
evoke_breadcrumb	PASS	PASS	PASS
evoke_kv_restore	PASS	PASS	PASS
evoke_attention	PASS	PASS	PASS
infllm	PASS	PASS	PASS

Table 3: Agent-bench probe outcome on Qwen 2.5 7B, `block_size=64`, three KV budgets. Every recovery-less policy fails the probe at every budget; every recovery-bearing policy passes. Llama 3.1 8B reproduces the same divide. EVOKE strategies, breadcrumb, and InfLLM all PASS the planted-config probe at every budget; recency, streaming_llm, evoke_discard, h2o all fail.

additionally avoids recovery entirely at b=512: the multi-signal scorer assigns the config-file block a high enough score that eviction never picks it in the first place. Appendix A.1 isolates this differential.

Needle-in-a-haystack. `scripts/niah_bench.py` plants a distinctive fact (a needle, e.g., "the secret password for the laboratory vault is icarus-pinwheel-43") at a controlled depth inside a long synthetic haystack of paragraphs about diverse made-up topics. The haystack is deterministic from a fixed seed, with 40 paragraphs at `block_size=64` ($\sim$60 blocks, $\sim$4$\times$ the 1024-token budget). Five distinct needles cover four answer types; five depths cover {5, 25, 50, 75, 95}% of haystack position. Table 4 reports pass rate on Qwen 2.5 7B and Llama 3.1 8B at three budgets, 25 cells per (architecture, policy, budget).

	Qwen 2.5 7B			**Llama 3.1 8B**		
policy	b**=512**	b**=1024**	b**=2048**	b**=512**	b**=1024**	b**=2048**
recency	20 %	20 %	40 %	0 %	0 %	0 %
streaming_llm	0 %	20 %	20 %	12 %	20 %	24 %
evoke_discard	0 %	20 %	20 %	12 %	20 %	24 %
evoke_breadcrumb	0 %	20 %	20 %	12 %	20 %	24 %
h2o	20 %	20 %	40 %	20 %	20 %	40 %
snapkv	20 %	20 %	40 %	44 %	68 %	84 %
infllm	**96 %**	**96 %**	80 %	**84 %**	76 %	68 %
evoke_kv_restore	**96 %**	**100 %**	**100 %**	76 %	**88 %**	**88 %**
evoke_attention	**96 %**	**100 %**	**100 %**	76 %	**88 %**	**88 %**

Table 4: NIAH pass rate, 25 (needle, depth) cells per cell. Recovery-bearing policies (evoke variants, infllm) recover the needle across the full depth range. Recovery-less policies pass only at the recency-naturally-resident depth. SnapKV's heavy-hitter retention on Llama at looser budgets (44/68/84 %) is a documented single-needle exception that does not carry to multi-fact (Table 6).

Recovery-less policies all pass only at depth 95 % where the needle sits in the recent tail and never gets evicted; at every other depth they cannot recover the lost block. SnapKV on Llama is the one documented exception: its heavy-hitter selection preferentially retains the needle when the needle token has high attention mass during single-needle ingestion, climbing to 44/68/84 % across budgets. It falls back to the recovery-less cluster on the multi-fact bench in Section 7.4, where five sequential probes shift the model's attention across topics and no single block holds top-K mass throughout. The fail cells for evoke variants at b=512 (and InfLLM at b=512 and 1024) are the depth-95 capital cell where smart-recovery still fires on the resident needle and recovered blocks land as freshest context, anchoring the model on post-needle haystack rather than on the naturally-resident needle text; the K and V tensors are correctly spliced (`kv_block_load` runs successfully), but the recovery competes with what was already there.

Multi-fact (n=5 pilot). `scripts/multifact_bench.py` plants five distinct facts in the same session at jittered depths and probes all five in sequence at the end. Each per-fact probe runs its own smart-recovery pass, so the cache churns continuously through five different topic shifts. Five seeds vary the paraphrase template and the per-fact value. Scoring is hybrid: a string-match-set on multiple value variants is the primary metric, and a local LLM judge (`gemma-4-E4B-it` Q4_K_M, loaded sequentially after the SUT) decides ambiguous answers. Table 5 reports the n=5 pilot on Qwen 2.5 7B at b=1024, where the gap between recovery-bearing and recovery-less policies first appeared. The tighter n=15 extension follows in Section 7.4, where it surfaces the budget crossover against InfLLM.

strategy	pass rate	95 % Wilson CI
recency	4.0 %	[0.71 %, 19.54 %]
streaming_llm	0.0 %	[0.00 %, 13.32 %]
evoke_discard	0.0 %	[0.00 %, 13.32 %]
evoke_breadcrumb	0.0 %	[0.00 %, 13.32 %]
h2o	0.0 %	[0.00 %, 13.32 %]
snapkv	4.0 %	[0.71 %, 19.54 %]
evoke_attention	48.0 %	[30.03 %, 66.50 %]
evoke_kv_restore	**60.0 %**	**[40.74 %, 76.60 %]**
infllm	**64.0 %**	**[44.52 %, 79.75 %]**

Table 5: Multi-fact n=5 pilot on Qwen 2.5 7B at b=1024, 25 facts per row (5 facts × 5 seeds). Wilson 95 % confidence intervals. Per-seed evoke_kv_restore: {80, 60, 40, 60, 60} %; per-seed infllm: {80, 60, 40, 60, 80} %.

The structural divide reproduces on the cross-architecture sweep: on Qwen 3.5 9B (hybrid Mamba+Attention with thinking) EVOKE reaches 68 % [48.41, 82.80] while every recovery-less baseline stays in 0 to 8 %, and even H2O (the strongest non-EVOKE comparator) manages only 8 % [2.22, 24.97]. On Qwen 3.6 35B-A3B (MoE + thinking, IQ2 quant) EVOKE drops to 52 % [33.50, 69.97] while every baseline including H2O is at 0 %; the absolute pass-rate falls with quantization aggressiveness but a large absolute gap over the best baseline (52 to 60 points) holds across all three architectures (Figure 4).

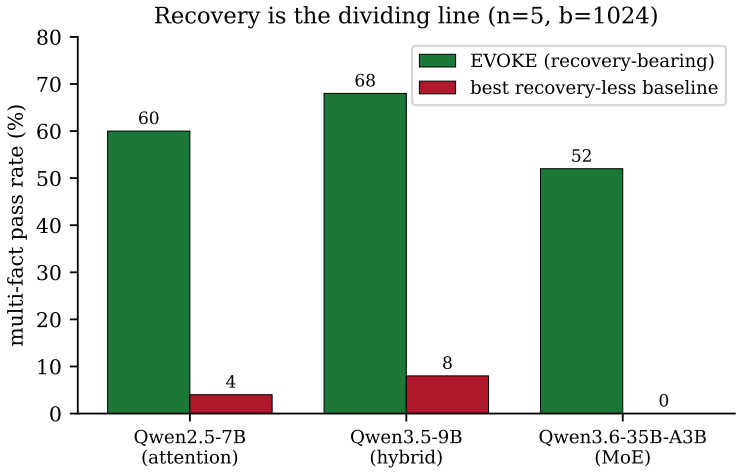

Figure 4: Recovery is the dividing line across architectures. EVOKE (recovery-bearing) passes multi-fact on pure attention (Qwen 2.5 7B), hybrid Mamba/Attention (Qwen 3.5 9B), and MoE (Qwen 3.6 35B-A3B); the best recovery-less baseline on each collapses to 0–8 % (n=5, b=1024).

7.4 InfLLM validates the thesis: two schedulers on the same substrate

Section 7.3 established the structural divide; the closest external-memory baseline (InfLLM, reimplemented on our substrate via `kv_block_load`) sits on our side of it. This subsection runs the

head-to-head between the two schedulers on the shared memory-hierarchy substrate. Both use identity-keyed recovery (the same `kv_block_load` paste-back of the model's own K and V), both treat eviction as a reversible cut, and both manage the cache as a hierarchy with an external-memory tier; they differ on scheduling. EVOKE evicts via a watermark policy with a multi-signal scorer (attention, recency, coherence, retrieval-embedding similarity) and recovers top-K=4 per turn; InfLLM evicts more aggressively (to a sinks-plus-25 %-local-window footprint) and recovers a wider top-K=8 via pure-retrieval scoring.

The n=5 pilot in Section 7.3 clusters the three recovery-bearing policies between 48 and 64 percent with overlapping CIs. Extending the sweep to 15 seeds at three budgets (75 facts per cell, both fully-swept architectures), the tighter intervals reveal a budget crossover rather than one scheduler strictly dominating (Table 6, Figure 5).

strategy	b=**512**	b=**1024**	b=**2048**
Qwen 2.5 7B			
infllm	**81.3 % [71.1, 88.5]**	58.7 % [47.4, 69.1]	56.0 % [44.7, 66.7]
evoke_kv_restore	50.7 % [39.6, 61.7]	57.3 % [46.1, 67.9]	65.3 % [54.1, 75.1]
evoke_attention	49.3 % [38.3, 60.5]	58.7 % [47.4, 69.1]	**65.3 % [54.1, 75.1]**
Llama 3.1 8B			
infllm	**81.3 % [71.1, 88.5]**	65.3 % [54.1, 75.1]	56.0 % [44.7, 66.7]
evoke_kv_restore	58.7 % [47.4, 69.1]	57.3 % [46.1, 67.9]	65.3 % [54.1, 75.1]
evoke_attention	60.0 % [48.7, 70.3]	57.3 % [46.1, 67.9]	**66.7 % [55.4, 76.3]**
recency	0.0 % [0.0, 4.9]	0.0 % [0.0, 4.9]	0.0 % [0.0, 4.9]
streaming_llm	0.0 % [0.0, 4.9]	1.3 % [0.2, 7.2]	13.3 % [7.4, 22.8]
h2o	0.0 % [0.0, 4.9]	8.0 % [3.7, 16.4]	29.3 % [20.2, 40.4]
snapkv	1.3 % [0.2, 7.2]	16.0 % [9.4, 25.9]	42.7 % [32.1, 53.9]

Table 6: Multi-fact pass rate at n=15 (75 facts per cell) with Wilson 95 % CIs. Recovery-less baselines (bottom block) shown once for the Llama sweep; the Qwen sweep produces the same cluster ($\leq$25 % at the loosest budget). At b=512 InfLLM statistically separates from EVOKE on both architectures (non-overlapping Wilson CIs). At b=2048 EVOKE leads with overlapping CIs.

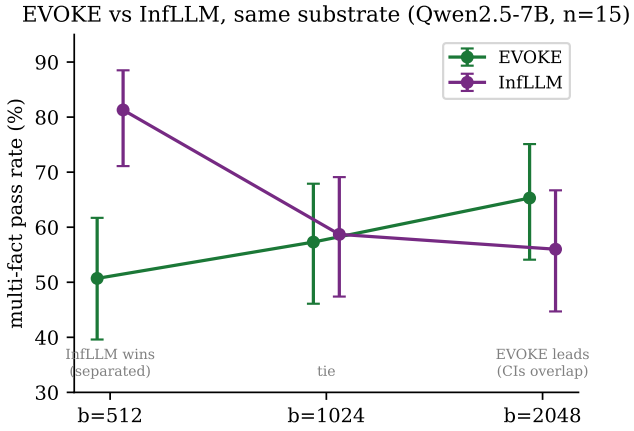

Figure 5: EVOKE versus InfLLM on the same recompute-free substrate (Qwen 2.5 7B, multi-fact, n=15, Wilson 95 % CIs). The two trade places along the budget axis: InfLLM's wider K=8 separates at b=512, the policies tie at b=1024, and EVOKE leads at b=2048 with overlapping CIs (directional, not separated).

EVOKE and InfLLM trade places along the budget axis. At b=512 InfLLM's larger K=8 retrieval catches more relevant blocks per turn and statistically separates from EVOKE on both architectures (Qwen non-overlapping CIs: [71.1, 88.5] vs evoke_kv_restore [39.6, 61.7]; Llama non-overlapping CIs: [71.1, 88.5] vs evoke_attention [48.7, 70.3]). At b=1024 the policies tie. At b=2048 EVOKE pulls ahead but with overlapping CIs, so we report this as directional rather than separated. Pure recovery-less baselines remain $\leq$25 % at the loosest budget with tight n=15 CIs.

Why InfLLM wins at tight budget. The per-fact failure cross-tab shows the "date" fact (a Treaty of Vrenholm probe) is structurally hard for every strategy ($\leq$1/5 across all nine strategies). EVOKE's $K{=}4$ selection misses 91 % of date cells while InfLLM's $K{=}8$ misses 67 %, indicating that the EVOKE-vs-InfLLM gap at tight budget is selection-failure-dominated, not substitution noise. Both policies use the same recompute-free recovery primitive; what InfLLM does better at tight budget is select a wider net of candidate blocks per recovery pass.

Why EVOKE wins at loose budget. At $b{=}2048$, headroom lets the attention-driven scorer keep more of the right blocks resident in the first place (the `evoke_attention` configuration is the top scorer on both architectures at the loosest budget). InfLLM's fixed aggressive footprint (sinks plus a 25 % local-window tail) does not adapt to budget headroom; EVOKE's watermark-driven eviction does. The Llama capture-layer ablation in Appendix A.9 reinforces this reading: changing the scorer's capture layer does not change the residency set on Llama at $b{=}1024$, because the workload bottlenecks on selection.

What the head-to-head means for the thesis. The crossover is a result *within* the thesis, not against it. Both schedulers are recovery-bearing; both clear the structural divide that separates them from recency, StreamingLLM, H2O, and SnapKV. The fact that InfLLM at $K{=}8$ beats EVOKE at $K{=}4$ on tight-budget multifact is a statement about per-turn recovery breadth, not about whether the cache should be paged virtual memory. Both schedulers answer that question the same way, and the data backs both. **What both schedulers share is the thesis: identity-keyed recovery, eviction-as-reversible-cut, cache-as-hierarchy. The scheduler choice is second-order, budget-dependent, and reported in full here so that practitioners pick the right K for their budget rather than read either scheduler as universally better.**

7.5 Clearing the context wall at scale

The recall benchmarks fit within the context window; the value of reversible eviction shows when the session does not. We drive a 66,147-token agentic session (150 keyed sections read in sequence, then re-referenced) under an 8,192-token KV budget with a 32,768-token context (`scripts/scale_demo.py`, Qwen 3-14B). Figure 6 contrasts three arms.

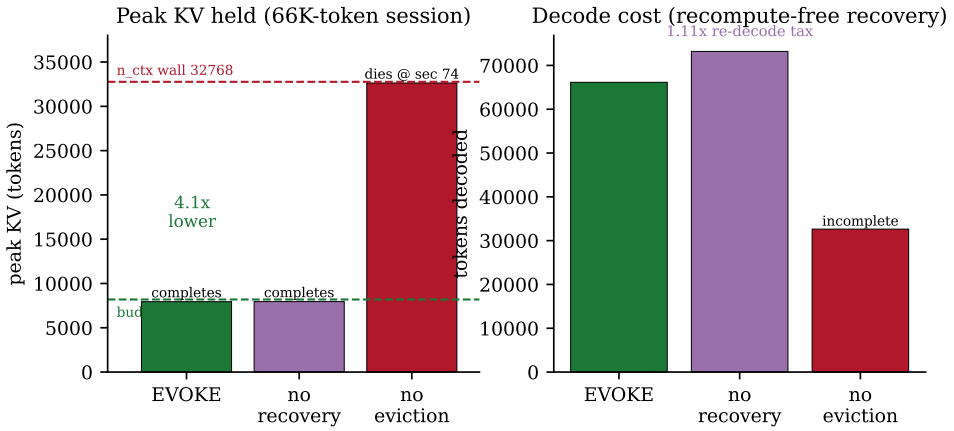

Figure 6: A 66K-token session under an 8K KV budget (Qwen 3-14B). Left: peak KV held. Without eviction the cache grows into the 32K context wall and the run dies mid-corpus (section 74 of 150); EVOKE holds the whole session within budget at $4.1\times$ lower peak KV. Right: tokens decoded. EVOKE recovers re-referenced sections recompute-free (decoded $\approx$ the corpus once), while the discard-only arm pays a $1.11\times$ re-decode tax on the re-references.

Without eviction the cache reaches the context wall at section 74 and the run cannot continue, the failure reversible eviction exists to prevent. EVOKE completes the full corpus with peak KV inside the budget (a $4.1\times$ reduction) and recovers re-referenced sections recompute-free; the discard-only arm stays in budget but re-decodes every re-reference. We make no within-window recall claim here (Section 7.2 covers fidelity).

7.6 Wall-clock head-to-head: parity with truncate, not advantage

We run the same 14-turn planted-fact session through three server policies via `scripts/baseline_bench.py`: `no_eviction` (high_watermark=0.999, budget lifted to `n_ctx`; what a vanilla OpenAI-compatible server with prefix caching does), `truncate` (StreamingLLM-style: drop the oldest non-sink block under budget pressure, evicted content gone), and `evoke` (default scoring with `kv_restore`). Qwen 2.5 7B, b=1024 where applicable, `n_ctx=16384`, n=15 runs, 95 % CI via t-distribution.

policy	budget	active end	evict	recov	probe	elapsed (95 % CI)
no_eviction	16384	2476	0	0	PASS	18.90s [18.74, 19.05]
truncate	1024	**685**	**70**	0	PASS	22.11s [19.46, 24.76]
evoke	1024	**684**	**90**	**24**	PASS	**21.20s [21.07, 21.33]**

Table 7: 14-turn planted-fact head-to-head on Qwen 2.5 7B, n=15. All 15 runs of every policy answer the probe correctly. EVOKE's [21.07, 21.33] interval lies inside truncate's [19.46, 24.76] interval; they are not statistically distinguishable at α=0.05.

The headline is recall and footprint parity with truncation, not a speed win. EVOKE and truncate both sit at 684 to 685 active tokens (roughly 3.6× smaller than no_eviction's 2476 footprint, inside the 1024-token budget), while no_eviction only fits because this 14-turn session is within `n_ctx`; a longer session crashes with no available KV slot. On wall-clock the two are not distinguishable at α=0.05 (truncate's variance is dominated by SSH-launch jitter). EVOKE's claim is recovery, not throughput: it matches no_eviction's recall at ∼3.6× smaller cache and at truncate-parity wall-clock while restoring the missing fact through 24 `kv_block_load` splices rather than per-turn re-prefill, which truncate cannot do. Here the planted fact happens to stay in truncate's recency window, so truncate passes; the agent-bench scaffold (Section 7.3) runs the same fact past recency and truncate fails at every budget. The session-length sweep (Appendix A.7) extends to $T \in \{28, 56\}$ turns and confirms the wall-clock gap stays within ∼10 % across the 4× scaling.

The no_eviction elapsed time is the natural latency floor for a stateful server: every turn's prefix matches the previous cache byte-for-byte and only the new tail decodes. It is what a paged-attention or prefix-caching server can hit when the cache has room; EVOKE's overhead over this floor (2.30 seconds, ∼12 %) is the cost of pretending the budget is 1024 instead of 16,384.

7.7 Substrate portability: what does not transfer to vLLM

The headline finding is negative. We ported the EVOKE primitive and policy layer to vLLM v1 with PagedAttention (Kwon et al., 2023) on a fork (`github.com/Anyesh/vllm`). The recovery primitive (paged byte transfer plus RoPE re-anchoring) composes from vLLM's existing `swap_blocks_batch` and `rotary_embedding` kernels with no CUDA-side work (the inverse-plus-forward rotation is a numerical no-op to within 1.6×10^{-2} bf16 max absolute deviation) and `EvokeCachePolicy` registers as a first-class offload-tier policy. The policy layer does not land: similarity-based smart-recovery needs a session-scoped logical position space for the recovered bytes, and the V1 scheduler has none (it treats the cache as a contiguous prefix of the active request, with no position space that outlives a single `generate()` call). llama.cpp provides that slot natively; vLLM v1 does not.

What *does* transfer is same-content gap-fill: when a multi-turn agent re-sends growing history, prefix-cache hashes match previously offloaded blocks and the offload-tier eviction policy governs which prior blocks survive. A one-cell probe (`scripts/niah_vllm_samecontent.py`, Qwen 2.5 7B, EVOKE vs LRU offload policy) scores 5/5 in both arms: the needle text is in the re-sent prompt, so the metric cannot distinguish the policies (n=5, no CIs). The finding names a concrete precondition (a session-scoped logical position space) and a substrate that lacks it; the follow-up in Section 9 is a host-side session abstraction, not more port engineering.

7.8 Re-anchoring recovered blocks: when tail placement helps

The recovery splice re-anchors a recovered block to a new contiguous tail position (Section 3); ArkVale (Chen et al., 2024) instead recalls evicted KV at its original absolute position. To isolate that one difference we force-evict and force-recover 12 facts spread across the context and vary only

where the recovered bytes land: `compact` re-anchors to the tail (EVOKE), `sparse` splices back at the original position (ArkVale's placement). Two never-evicted controls bound the result: `no_evict` leaves the facts at their original positions, `no_evict_tail` decodes them natively at the tail. Qwen 2.5 7B, 128K context, 8 seeds (Table 8); `scripts/multifact_position_bench.py`.

distance (tokens)	compact (tail)	sparse = no_evict (orig. pos.)	no_evict_tail (native)
16,384	1.00	1.00	1.00
81,920	0.97	0.70	1.00
98,304	0.83	0.74	1.00
114,688	0.77	0.65	1.00

Table 8: Multi-fact recall by where recovered blocks are placed (Qwen 2.5 7B, 12 facts, 8 seeds). `sparse` reproduces `no_evict` exactly, so original-position recovery is lossless. `compact` beats `sparse` only at 80K+, and stays below the native `no_evict_tail` ceiling at every distance.

Three readings. First, `sparse` matches `no_evict` to two decimals at every distance, so original-position recovery is lossless and the baseline is not weakened. Second, at and below 64K all arms are equal: re-anchoring earns nothing where the model handles the distance natively; at 80K+ `compact` beats `sparse` (0.97 vs 0.70 at 82K), so tail placement helps near the context edge. Third, `compact` stays below `no_evict_tail` (a flat 1.00), so re-anchoring is lossy relative to decoding the same facts natively at the tail; the gap is the recovered block's K and V still encoding the context it attended at its original position, not a RoPE error (positions are clean and the misses do not track the re-anchor distance).

The benefit is also narrow. Repeating the sweep on Qwen 2.5 3B in its 32K window (`multifact_3b_incontext.json`) reverses the sign: `compact` drops to 0.86 to 0.92 while `sparse` and both controls hold at 1.00, because a 32K model never reaches the far-position distances where tail placement pays, so re-anchoring is pure cost. The likely mechanism is positional, lost-in-the-middle bias (Liu et al., 2023b; Hsieh et al., 2024): moving content into the well-attended tail dodges the model's far-position recall weakness rather than improving KV fidelity. Tail re-anchoring therefore helps only near a long-context model's far edge, is neutral at normal distances, and hurts on short-context models; it is not a free improvement over ArkVale's original-position recall.

8 Discussion and Limitations

Hybrid memory, thinking mode, and pure SSM. On hybrid models, Qwen 3.5's `<think>` trace is stripped from the response but kept in the physical cache. Strict alignment would evict the thinking range from attention, but `llama_memory_hybrid::seq_rm` rejects partial tail rollback of the recurrent half (a Mamba state cannot be sliced) and aborts, so the session resets on the divergent turn (one such reset in the Qwen 3.5 demo at filler 9). The shipping workaround is `EVOKE_SUPPRESS_THINKING_STRIP=1`: the trace stays in returned content, the client echoes it back, and the cached state stays aligned. A future no-shift eviction mode (attention-only `seq_rm`/`seq_add`, already in the fork) would drop the thinking range from attention while leaving the recurrent state untouched. EVOKE operates only on the attention component; pure-SSM models (Mamba, RWKV) are out of scope as they have no KV cache, and a hybrid model's blurry SSM state is preserved because EVOKE does not touch it.

Memory accounting and deployment realism. Saved blocks live in host RAM, and the cost is real. For Qwen 2.5 7B at `block_size=128`, one cell costs 56 KiB (28 layers $\times$ 2 $\times$ 512 `n_embd_kv_gqa` $\times$ 2 bytes), one block is 7 MiB, and a 1000-block session is roughly 7 GiB; N co-located sessions pay $N \times 7$ GiB, scaling up on Llama 3.1 8B with its larger footprint ($\sim$131 KB/token vs Qwen's $\sim$60). On a single-tenant box ($\sim$32 to 64 GiB host RAM) the tier is binding at $\sim$5 to 9 concurrent sessions before `kv_restore_ram_budget_bytes` demotes to disk spill (Appendix A.5 measures degradation under this pressure). Both the LRU bound and the disk-spill tier (`kv_restore_spill_path`) are off by default; multi-tenant operators should enable at least the LRU bound. EVOKE trades host RAM proportional to evicted content for exact recovery; summarizing schemes (compressive transformers (Rae et al., 2019)) trade the reverse. The session pool (Section 5) swaps KV state but keeps model weights as one VRAM instance, so concurrency is bounded by per-session KV footprint: a 7B Q4_K_M model takes $\sim$5 GB, leaving $\sim$11 GB on a 16 GB GPU for

active KV, swap-in state, and FA working memory. Lifting that ceiling needs tensor-parallel weight sharing or thinner per-session quantization.

Smart-recovery is a small retrieval system, by design. Policy-layer quality is bounded by the retrieval encoder's discrimination on the workload: a poor encoder picks the wrong blocks to recover, even though the spliced-back bytes are still the model's own K and V. The line between identity-keyed substitution and similarity-shaped selection is drawn once in Section 4.

Contribution boundary: the recovery primitive. The $n{=}15$ multi-fact sweep (Section 7.4) and its Llama 3.1 8B replication (Appendix A.9) flip the EVOKE-vs-InfLLM ranking along the budget axis: at tight budgets the win goes to whichever policy makes fewer selection failures, and InfLLM's $K{=}8$ pure-retrieval catches more blocks per turn than EVOKE's $K{=}4$ (the per-fact failure cross-tab confirms the gap is selection-dominated, not substitution noise), while at loose budgets headroom lets the attention-driven EVOKE scorer take the top spot. The contribution is therefore the recompute-free identity-keyed K/V splice that divides recovery-bearing from recovery-less policies (20 to 40 % versus 76 to 100 % on NIAH, 0 % versus 50 to 80 % on multifact); the attention scorer is a budget-dependent improvement on top, not the central claim. The Llama capture-layer ablation (Appendix A.9) reinforces this: changing the capture layer does not move the Llama residency set at budget 1024 because the workload bottlenecks on selection.

An ArkVale-style recall ablation, and why it is not a head-to-head. We built an ArkVale-style recall policy on our substrate (fork primitives `llama_query_capture` and `llama_key_capture` expose the per-decode query and keys at a scoring layer). Each block carries a key bounding-volume digest (per-dimension min/max and mean); a recovered block is scored by a cuboid-mean estimate of $q \cdot k$ from the captured query, and a bounded top-K recall-and-evict keeps the highest-scoring blocks resident at their original positions. On multi-fact at Qwen 2.5 7B it trails both embedding-recall arms at every budget (Table 9). Two axes separate them, both favoring EVOKE's choices on this task: importance-estimate recall over key digests versus embedding-similarity recall, and original-position placement versus the compact tail re-anchor (Section 7.8).

recall policy	b**=512**	b**=1024**	b**=2048**
ArkVale-style (cuboid-mean, original-position)	0 %	4 %	20 %
`evoke_kv_restore` (embedding, compact)	52 %	60 %	64 %
`infllm` (embedding $K{=}8$, compact)	88 %	64 %	60 %

Table 9: ArkVale-style recall ablation: multi-fact pass rate on Qwen 2.5 7B ($n{=}5$, 25 facts per cell). The ArkVale-style arm (cuboid-mean importance recall at original positions, single scoring layer) trails the embedding-recall arms at every budget; it reaches 70 % at $b{=}4096$, where the document mostly fits, so the tight-budget gap is recall selection and placement, not a broken pipeline. This isolates the recall-selection and placement axes; it is not a reproduction of ArkVale's per-layer paging (Section 8).

This ablates those two axes, not a reproduction of ArkVale. ArkVale selects pages *per layer* (each layer holds its own resident set), which a whole-sequence cache (one `seq_rm` drops a position across all layers) cannot represent, and our arm scores on a single layer. The arm runs correctly at loose budget (70% at $b{=}4096$, where the document mostly fits), so the tight-budget gap is selection and placement, not a broken pipeline. We make no claim of advantage over ArkVale's full method; EVOKE's contribution is the agent-memory setting, the cross-architecture reach, and the measured recovery fidelity of Section 7.2.

Cross-architecture latency depth. Sections 7.1, 7.3 and 7.6 and appendices A.1, A.6 and A.7 measure latency and scorer ablations on Qwen 2.5 7B, and Appendix A.9 adds an end-to-end Llama 3.1 8B run (NIAH, multifact, agent_bench). The per-token K/V footprint scales predictably (layers $\times$ heads $\times$ head dim $\times$ 2 $\times$ element width), so the host-to-GPU memcpy that dominates `kv_block_load` at long blocks shifts in a known direction with parameter count, GQA grouping, and quantization width; the cost model predicts where the band moves for unmeasured substrates, but only two are measured end-to-end here.

Embedding quality and recovery placement. Smart-recovery scoring uses `bge-small-en-v1.5` (384-dim), sufficient for the NIAH workload (Section 7.3) and swappable via `EvokeConfig.use_retrieval_embeddings`; larger or task-tuned encoders are an unrun sweep. Recovery currently appends blocks at the cache tail before the new user message decodes, so they land as earlier context the freshest probe attends back to, which fits the chat-completions pattern. For streaming workloads where the model generates tokens that then need earlier context, an in-place insertion path (the attention-only `seq_rm`/`seq_add` primitives) would slot recovered blocks back at their original logical position.

9 Future Work

- **Session abstraction for paged substrates.** Section 7.7 identifies the missing vLLM v1 ingredient: a logical position space that outlives a single request. The open question is whether a host-side session abstraction (a `session_id` on `Request` with cache scoped to it) can be added without breaking the V1 scheduler's contiguous-prefix invariants, and how it composes with SGLang's RadixAttention (Zheng et al., 2024) and LMCache (LMCache Team, 2024). The transfer mechanism (paged scatter/gather) already exists via `swap_blocks_batch`; the work is the scheduler-side abstraction that gives transfers a session-scoped destination.
- **iSWA end-to-end on Gemma.** `llama_kv_block_save/load` now handle both base and SWA sub-caches via `llama_get_kv_cache_swa()` (fork commit `1f37e75e7`); the remaining engine-side dual-cache bookkeeping and a Gemma 3 or 4 GGUF end-to-end smoke are the next step.
- **No-shift eviction mode for hybrid plus thinking.** Use the attention-only `llama_kv_block_seq_rm/seq_add` primitives (already in the fork) to evict attention cells without compacting, keeping a hybrid model's recurrent state in sync while the thinking range drops from attention.
- **Multi-layer scorer.** The fork exposes `llama_attn_capture_set_layers` for up to 16 layers per decode; the current scorer uses a single mid-layer, where averaging an early, middle, and late layer could pick up both syntactic and semantic signals.
- **Tensor-parallel and quantized variants.** Validate the C primitives unchanged across multi-GPU tensor-parallel layouts and the full Q3 to Q8 range.
- **Continuous learning of harness priorities.** `evoke_priority` is harness-set today; EVOKE could instead learn priority from observed attention so harnesses that set none still benefit.
- **Multi-fact, judge-scored, randomized-position bench.** Section 7.4 exercises five facts per session with a model judge; lift this to a RULER- or LongBench-style suite with per-position jitter and partial-credit scoring so the metric is not a binary single-probe hit.
- **Real-agent traces.** The benches here are synthetic. A head-to-head on a real workload (SWE-bench Verified, τ-bench, or a recorded Claude Code session) would remove the constructed cache pressure and probe shape. EVOKE's server is already wire-compatible with the chat-completions API those harnesses use, so the work is the eval setup, not a system port; we decline to extrapolate the synthetic results onto it.
- **KV-quantization head-to-head against KIVI.** Appendix A.8 measures Q4_0 only. KIVI's per-channel-per-token asymmetric scheme keeps generation viable at low bit-widths and is the experiment that would settle the "why not just quantize?" question for production quantization.

10 Conclusion

For a long-running agent, EVOKE makes the KV cache an OS-style memory hierarchy: cold blocks live in host RAM at metadata cost, and a recompute-free splice brings them back to GPU through a single RoPE rotation when the agent refers to their positions again. The restored tensors are the same bytes the model first computed, and a controlled probe confirms a recovered block is as usable for

recall as a fresh re-decode (on pure-attention and hybrid models, and under heavy eviction churn) so reversible eviction reclaims the budget without losing recall.

All numbers below are from a single 16 GB consumer GPU running the four model families. On Qwen 2.5 7B, the `kv_block_load` primitive runs in 0.5 to 7 ms across block sizes from 20 to 1280 tokens, 20 to 32 times faster than re-prefilling the same tokens through the model (5.9 to 7.5$\times$ over the full save+load lifecycle, factoring in the eviction-side save cost). On Llama 3.1 8B the bands are 1.78 to 15.66 ms and 7 to 15$\times$ load-only (2.6 to 2.8$\times$ lifecycle). On a 14-turn planted-fact session (n=15), EVOKE matches the unconstrained `no_eviction` baseline's recall at $\sim$3.6$\times$ smaller cache footprint and at `truncate`-parity wall-clock (21.20 s [21.07, 21.33] vs 22.11 s [19.46, 24.76]; `truncate`'s SSH-launch jitter dominates its CI so the two are not statistically distinguishable at α=0.05). At scale, EVOKE holds a 66K-token agent session in an 8K KV budget at $4.1\times$ lower peak KV, where a no-eviction run hits the 32K context wall and cannot complete; and the stateful session pool sustains 16 concurrent agent sessions with every recall probe passing (80/80) and a p99-to-p999 tail spread under 0.13 s.

Across the NIAH, multi-fact, and agent_bench head-to-heads, every recovery-less baseline (recency, StreamingLLM, H2O, SnapKV) stays at or below 43 % multi-fact pass rate while every recovery-bearing policy (`evoke_kv_restore`, `evoke_attention`, and the same-substrate `infllm` adaptation) reaches at least 48 %, with the same divide on single-needle NIAH and categorical pass/fail on agent_bench (SnapKV's heavy-hitter retention on Llama at looser budgets is the only documented exception, Appendix A.9). The partition holds at b=1024 on hybrid Mamba/Attention Qwen 3.5 9B (68 %) and MoE Qwen 3.6 35B-A3B (52 %). Within the recovery-bearing cluster, a 15-seed multi-fact sweep against InfLLM at K=8 shows a budget-axis crossover: InfLLM wins at b=512 (non-overlapping Wilson CIs on both architectures), the two tie at b=1024, and EVOKE's point estimates lead at b=2048 (CIs overlap, Section 7.4). Both schedulers depend on the same recovery splice, the core contribution; scheduler choice and its K are parameters tuned to budget. The attention-driven scorer is an additional gain on top of the primitive when budget headroom allows.

Code and Reproducibility

The EVOKE policy layer (Python) is at `github.com/Anyesh/EVOKE` under Apache 2.0. The forked llama.cpp with the C primitives is at `github.com/Anyesh/llama.cpp` under MIT (the upstream license). The partial vLLM port referenced in Section 9 is at `github.com/Anyesh/vllm` under Apache 2.0 (the upstream vLLM license). Every number in Section 7 was produced by a script in `scripts/` checked into the EVOKE repository, with the exact invocations listed in Appendix B. Raw output files for the agentic eval and the keepalive eval are checked in under `results/`.

References

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention. *SOSP 2023*.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*.

Liu, Z., Desai, A., Liao, F., Wang, W., Xie, V., Xu, Z., Kyrillidis, A., & Shrivastava, A. (2023). Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time. *NeurIPS 2023*.

Rae, J. W., Potapenko, A., Jayakumar, S. M., & Lillicrap, T. P. (2019). Compressive Transformers for Long-Range Sequence Modelling. *arXiv preprint arXiv:1911.05507*.

Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., & Liu, Y. (2024). RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing 568:127063, 2024*.

Xiao, G., Tian, Y., Chen, B., Han, S., & Lewis, M. (2024). Efficient Streaming Language Models with Attention Sinks. *ICLR 2024*.

Xiao, C., Zhang, P., Han, X., Xiao, G., Lin, Y., Zhang, Z., Liu, Z., Han, S., & Sun, M. (2024). InfLLM: Training-Free Long-Context Extrapolation for LLMs with an Efficient Context Memory. *arXiv preprint arXiv:2402.04617*.

Chen, R., Wang, Z., Cao, B., Wu, T., Zheng, S., Li, X., Wei, X., Yan, S., Li, M., & Liang, Y. (2024). ArkVale: Efficient Generative LLM Inference with Recallable Key-Value Eviction. *NeurIPS 2024*.

Lee, K.-H., Park, E., Han, D., & Na, S.-H. (2025). CacheFocus: Dynamic Cache Re-Positioning for Efficient Retrieval-Augmented Generation. *arXiv preprint arXiv:2502.11101*.

Hsieh, C.-Y., Chuang, Y.-S., Li, C.-L., Wang, Z., Le, L. T., Kumar, A., Glass, J., Ratner, A., Lee, C.-Y., Krishna, R., & Pfister, T. (2024). Found in the Middle: Calibrating Positional Attention Bias Improves Long Context Utilization. *Findings of ACL 2024*.

Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2023). MemGPT: Towards LLMs as Operating Systems. *arXiv preprint arXiv:2310.08560*.

Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Re, C., Barrett, C., Wang, Z., & Chen, B. (2023). H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. *NeurIPS 2023*.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., & Sheng, Y. (2024). SGLang: Efficient Execution of Structured Language Model Programs. *NeurIPS 2024*.

Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., & Chen, D. (2024). SnapKV: LLM Knows What You are Looking for Before Generation. *NeurIPS 2024*.

Feng, Y., Lv, J., Cao, Y., Xie, X., & Zhou, S. K. (2024). Ada-KV: Optimizing KV Cache Eviction by Adaptive Budget Allocation for Efficient LLM Inference. *arXiv preprint arXiv:2407.11550*.

Cheng, J., Liu, Y., Wang, K., Xiang, X., Yu, X., Liu, J., Yi, F., & Jiang, J. (2024). LMCache: 7x Throughput Improvement Through Layered, Cross-Engine KV Cache. *vLLM Production Stack project*.

Gao, B., He, Z., Sharma, P., Kang, Q., Jevdjic, D., Deng, J., Yang, X., Yu, Z., & Zuo, P. (2024). Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention. *USENIX ATC 2024*.

Liu, Y., Li, H., Cheng, Y., Ray, S., Huang, Y., Zhang, Q., Du, K., Yao, J., Lu, S., Ananthanarayanan, G., Maire, M., Hoffmann, H., Holtzman, A., & Jiang, J. (2024). CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. *SIGCOMM 2024*.

Yao, J., Li, H., Liu, Y., Ray, S., Cheng, Y., Zhang, Q., Du, K., Lu, S., & Jiang, J. (2025). CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. *EuroSys 2025*.

Lin, B., Zhang, C., Peng, T., Zhao, H., Xiao, W., Sun, M., Liu, A., Zhang, Z., Li, L., Qiu, X., Li, S., Ji, Z., Xie, T., Li, Y., & Lin, W. (2024). Infinite-LLM: Efficient LLM Service for Long Context with DistAttention and Distributed KVCache. *arXiv preprint arXiv:2401.02669*.

Xiao, S., Liu, Z., Zhang, P., Muennighoff, N., Lian, D., & Nie, J.-Y. (2024). C-Pack: Packed Resources For General Chinese Embeddings. *SIGIR 2024*.

Gu, A., & Dao, T. (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. *arXiv preprint arXiv:2312.00752*.

Hsieh, C.-P., Sun, S., Kriman, S., Acharya, S., Rekesh, D., Jia, F., Zhang, Y., & Ginsburg, B. (2024). RULER: What's the Real Context Size of Your Long-Context Language Models? *COLM 2024*.

Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., & Li, J. (2023). LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. *ACL 2024*.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2023). Lost in the Middle: How Language Models Use Long Contexts. *TACL 2024*.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *NeurIPS 2022*.

Dao, T. (2023). FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arXiv preprint arXiv:2307.08691*.

Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., & Hu, X. (2024). KIVI: A Tuning-Free Asymmetric 2-bit Quantization for KV Cache. *ICML 2024*.

Cai, Z., Zhang, Y., Gao, B., Liu, Y., Liu, T., Lu, K., Xiong, W., Dong, Y., Chang, B., Hu, J., & Xiao, W. (2024). PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling. *arXiv preprint arXiv:2406.02069*.

Tang, J., Zhao, Y., Zhu, K., Xiao, G., Kasikci, B., & Han, S. (2024). Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference. *ICML 2024*.

Xiao, G., Tang, J., Zuo, J., Guo, J., Yang, S., Tang, H., Fu, Y., & Han, S. (2024). DuoAttention: Efficient Long-Context LLM Inference with Retrieval and Streaming Heads. *arXiv preprint arXiv:2410.10819*.

Prabhu, R., Nayak, A., Mohan, J., Ramjee, R., & Panwar, A. (2024). vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. *arXiv preprint arXiv:2405.04437*.

Qwen Team. (2024). Qwen 2.5 Technical Report. *arXiv preprint arXiv:2412.15115*.

Grattafiori, A., et al. (2024). The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*.

A Detailed Evaluations

This appendix collects evaluations that are referenced from the main body but whose full detail would crowd the headline results.

A.1 Per-cell agent_bench (scorer ablation context)

The full per-cell agent_bench table (referenced in Sections 7.3 and 7.6), with eviction counts and per-recovery latency, on Qwen 2.5 7B at `block_size=64`:

The `evoke_attention` row at b=512 is the differential against the heuristic scorer at the tightest budget: with attention scoring, the planted fact block is never evicted in the first place (the model's attention across the unrelated filler turns keeps landing on the config-file block, so the scorer assigns it a high score and eviction picks lower-attended document blocks instead). Total evictions drop by one (35 to 34) and recoveries go to zero, and the probe still passes. At 1024 and 2048 the two strategies converge: recency is sufficient when the budget can hold most history naturally.

A.2 Server eviction demo

A 14-turn session driven directly against the chat-completions API. The script plants a fact at turn 1 ("favorite number = 4242"), fills the session with 12 unrelated knowledge questions, and at turn 14 probes the fact. The recorded demos are kept as visual proof of the per-turn evolution of `active_tokens` against the budget.

Qwen 2.5 7B (pure attention), budget=1024. With smart top-K recovery and the `KVRestoreBackend` RAM budget configured tight (so the LRU spill tier exercises in the same run), the session survives **40 evictions and 13 recoveries** while the model still recalls "4242" at the probe.

Qwen 3.5 9B (hybrid Mamba+Attention with mrope and thinking), budget=1024. The model emits a `<think>...</think>` trace each turn, so per-turn token counts are higher. Hybrid memory rejects partial tail rollback of the recurrent half, which means strict thinking-strip evictions would force a session reset on every turn. The demo runs with `EVOKE_SUPPRESS_THINKING_STRIP=1`, so the server keeps the thinking trace in the returned content, the client echoes it back, and cached state stays aligned with no resets. The session settles at **26 evictions and 4 recoveries**, fact recalled.

budget	strategy	probe	evict	recov	rec_ms
512	recency	fail	35	0	0.00
512	streaming_llm	fail	35	0	0.00
512	evoke_discard	fail	35	0	0.00
512	h2o	fail	34	0	0.00
512	snapkv	fail	34	0	0.00
512	evoke_breadcrumb	PASS	35	0	17.25
512	evoke_kv_restore	PASS	35	1	**3.13**
512	evoke_attention	PASS	31	0	**0.01**
512	infllm	PASS	39	1	3.12
1024	recency	fail	29	0	0.00
1024	streaming_llm	fail	28	0	0.00
1024	evoke_discard	fail	28	0	0.00
1024	h2o	fail	24	0	0.00
1024	snapkv	fail	24	0	0.00
1024	evoke_breadcrumb	PASS	28	0	15.40
1024	evoke_kv_restore	PASS	28	1	**3.86**
1024	evoke_attention	PASS	26	1	3.75
1024	infllm	PASS	35	1	3.76
2048	recency	fail	9	0	0.00
2048	streaming_llm	fail	10	0	0.00
2048	evoke_discard	fail	10	0	0.00
2048	h2o	fail	4	0	0.00
2048	snapkv	fail	4	0	0.00
2048	evoke_breadcrumb	PASS	10	0	15.70
2048	evoke_kv_restore	PASS	10	1	**3.89**
2048	evoke_attention	PASS	8	0	**0.00**
2048	infllm	PASS	31	1	3.07

A.3 Default-knob ablations

We sweep one policy knob at a time across the full 25-cell NIAH grid (5 needles $\times$ 5 depths) at b=1024 on Qwen 2.5 7B, holding everything else at the recommended `evoke_attention` configuration:

knob	swept values	pass rate per value
`block_size`	{32, 64, 128, 256, 512}	84, **100**, 80, 56, 20 %
`attention_capture_layer`	{4, 10, 14, 20, 24, 27}	all 100 %
`smart_recover_k`	{1, 2, 4, 8, 16}	all 100 %
`w_coherence`	{0.0, 0.2, 0.4, 0.6, 0.8}	all 100 %
`recent_tail_protect_frac`	{0.0, 0.05, 0.1, 0.2, 0.3}	all 100 %

The `block_size` curve is the only knob that moves the pass rate: `block_size=64` hits the headline 100 % pass rate, and degradation is symmetric on both sides. Finer-grained blocks (32) fragment fact-bearing content across two neighbouring blocks; coarser blocks (`128, 256, 512`) dilute the block embedding's representative signal with adjacent haystack content. The other four knobs are flat at 100 % on this workload; tighter-budget or multi-fact workloads (Section 7.4) where eviction pressure is sustained throughout the session would likely surface knob-level differentiation.

A.4 Smart-recovery policy factorial

The smart-recovery loop in Section 5 has three named policy decisions: (K1) score using a dedicated retrieval-tuned embedder applied to the raw user-message text, versus pooling LM hidden states; (K2) run the recovery splice before the new user-message tail is decoded, versus after; (K3) gate the top-k recovery against the strongest non-current-turn resident block's similarity, versus skipping. We run the full 2^3 factorial on NIAH at b=512 on Qwen 2.5 7B (25 cells per row).

Marginals over n=100: K1 (raw-text retrieval embedding) +72 pp (14 % off vs 86 % on); K2 (recover-before-decode) -4 pp main effect but +20 pp conditional on K1 on, -28 pp conditional on K1 off; K3 (resident-gate) 0 pp on this benchmark. K1 is the dominant policy decision by a wide margin: the

cell	K1	K2	K3	pass	Wilson 95 % CI
000	off	off	off	28.0 %	[14.3, 47.6]
001	off	off	on	28.0 %	[14.3, 47.6]
010	off	on	off	0.0 %	[0.0, 13.3]
011	off	on	on	0.0 %	[0.0, 13.3]
100	on	off	off	76.0 %	[56.6, 88.5]
101	on	off	on	76.0 %	[56.6, 88.5]
110	on	on	off	96.0 %	[80.5, 99.3]
111	on	on	on	96.0 %	[80.5, 99.3]

bge-small retrieval embedder widens the cosine band from 0.85 to 0.93 (LM hidden states) to 0.4 to 0.9, and the top-k selection becomes discriminative enough to actually pick the needle block over haystack noise. K2's asymmetry has a mechanical explanation: recovering before decode splices the recovered block earlier in the cache than the probe, so the model attends back to it; this is a win when retrieval is good (K1 on) and a regression when retrieval is bad (K1 off, the wrong block lands as earlier context). K3 contributes no measurable effect on this benchmark at b=512. The resident-gate was added to address a depth-95 % failure mode where a top-k recovery brought back a weaker match than the already-resident needle, but this factorial's metric does not expose that mechanism. At the budget and benchmark we ran the factorial against, K3 has no measurable effect.

A.5 Host-RAM pressure: graceful degradation

The four-tier saved-block pool (RAM-resident $\rightarrow$ disk-spilled $\rightarrow$ breadcrumb-only $\rightarrow$ discarded) is exercised under tight host memory by `scripts/hostram_bench.py`: a 20-fact session against an 80-paragraph haystack at b=1024 with `kv_restore_ram_budget_bytes` set to hold only 8 saved blocks ($\sim$29 MiB of K/V on Qwen 2.5 7B), with the disk-spill tier enabled.

Outcome: across the session, 207 blocks are evicted from active KV. 200 of them spill to disk, 4 stay RAM-resident at the end (the LRU pool), and 3 fall through to breadcrumb-only. No block is discarded outright; `lru_drops` = 0 because the spill tier catches every saved-pool eviction. Smart-recovery successfully restores 4 of 20 facts under this $\sim$14 % RAM ceiling, well below the $\sim$60 % rate the same workload would hit with unbounded RAM, but the system serves the entire session without crashing and the recovery pipeline reaches the correct stored bytes when it does pick them. The spill tier acts as the slow-path safety net rather than the dominant tier in production deployments configured with a larger RAM budget.

A.6 Persistent-reference (keepalive) ablation

The Appendix A.1 differential is observable only at the tightest budget because the filler turns in agent_bench are semantically unrelated to the planted fact: the model's attention drifts off the config-file block within a few turns and the two scoring policies converge.

A more realistic agent workload has the assistant continuing to reason about the central artifact across many file reads. We repeat the agentic eval with each of the ten filler-file contents referencing "the retry policy from config.py", and the final probe phrased "looking at the retry policy in config.py, what is the maximum retry limit?", a question whose answering tokens have strong attention to the early config block.

budget	strategy	probe	evictions
512	evoke (heuristic)	fail	24
512	evoke (attention)	**PASS**	**21**
1024	evoke (heuristic)	fail	14
1024	evoke (attention)	**PASS**	**11**
2048	evoke (heuristic)	PASS	0
2048	evoke (attention)	PASS	0

This bench is a scorer-only ablation: the harness does not invoke recovery. A block the scorer chooses to evict is gone for the rest of the run. Under that constraint, the attention path passes the probe at the

two budgets where the heuristic path fails outright. Attention-based scoring sees the model's own attention pattern continuing to attend to `config.py` through every filler turn and refuses to evict it. The heuristic-only scorer evicts the config block under pressure and the fact is gone before the probe runs. Recompute-free recovery is a second line of defense, but it works best when the eviction policy did not drop the wrong block in the first place.

A.7 Session-length scaling and the recovery-aware fix

The Section 7.6 head-to-head is fixed at 14 turns. The scaling sweep runs the same workload at $T \in \{14, 28, 56\}$ turns, 5 seeds per cell, and produces a paired curve: a baseline curve with the production scorer, and a recovery-aware curve with `w_recovery=1.0`.

turns	policy	active	evict	recov	elapsed (s, 95 % CI)
14	no_eviction	2476	0	0	19.19 [18.76, 19.63]
14	truncate	685	66	0	21.33 [20.76, 21.90]
14	evoke (baseline)	684	90	24	21.58 [21.17, 21.99]
14	evoke (recovery-aware)	683	65	4	21.79 [21.54, 22.04]
28	no_eviction	6221	0	0	51.05 [50.10, 52.00]
28	truncate	717	486	0	65.14 [64.35, 65.94]
28	evoke (baseline)	721	566	80	68.06 [67.23, 68.89]
28	evoke (recovery-aware)	616	507	20	67.84 [66.65, 69.03]
56	no_eviction	12607	0	0	106.11 [105.19, 107.03]
56	truncate	674	2316	0	170.41 [169.27, 171.54]
56	evoke (baseline)	664	2522	192	182.36 [180.67, 184.06]
56	evoke (recovery-aware)	675	2400	20	185.73 [184.98, 186.48]

EVOKE (either curve) holds the active-token footprint at $\sim$670 to 720 tokens across the $4\times$ session-length sweep, while `no_eviction` grows linearly to 12 607 tokens at $T{=}56$. Wall-clock is comparable to truncate within roughly 7 to 10 % across the sweep, not faster. We do not claim a speed win at long sessions; recovery has a cost and truncate has nothing to recover. EVOKE matches truncate's memory footprint and adds a recovery capability truncate does not have.

The recovery-aware fix (`w_recovery` = 1.0) shuts down a recover-then-immediately-evict thrash: recoveries drop from 24/80/192 to 4/20/20 across $T{=}14/28/56$ (-83 to -90 %), and evictions drop in lockstep. Wall-clock does not improve because the retrieval embedder ($\sim$10 ms per query) substitutes for the saved thrash work at these session lengths. The fix is transparent on accuracy: multifact verification shows recovery-aware 48/56/64 % vs kv_restore 52/60/64 % at budgets 512/1024/2048, CIs heavily overlap. The wall-clock parity is structural.

A.8 KV cache quantization on the measured substrate

A reviewer asked whether KV-cache quantization is the cheaper alternative to eviction. We measured llama.cpp's stock `GGML_TYPE_Q4_0` (per-tensor symmetric 4-bit), which is the production-available default in the runtime EVOKE targets. We did not run KIVI (Liu et al., 2024) or other per-channel-per-token asymmetric schemes; the result below speaks only to the Q4_0 substrate.

The Q4_0 result is uniform: NIAH, multifact, and agent_bench all collapse to 0 % at every budget. Representative agent_bench generation at $b{=}2048$ with full 5962-token context preserved at Q4 precision: `‘’, and, and, and, and, and, and, and, and...’’`. F16 EVOKE at $b{=}1024$ retains 96 to 100 % NIAH on the same workload. On this substrate (Q4_0, Qwen 2.5 7B) 4-bit KV quantization is not a usable alternative to eviction. We do not extrapolate this to KIVI or other per-channel asymmetric schemes, whose per-channel structure is specifically designed to keep generation viable at lower bit-widths; running them is the next experiment for a deployer choosing between policy-layer eviction and aggressive KV quant.

A.9 Llama 3.1 8B: cross-architecture detail

NIAH and multifact pass-rates on Llama 3.1 8B are included in the main tables (Tables 4 and 6). A capture-layer ablation on the same NIAH grid (`EVOKE_ABL_DIM=attention_layer`, values $\{8, 16, 20, 24, 28\}$) yields identical pass-rates at both budgets: 88 % [70.04, 95.83] (22/25 cells)

across all five capture depths at b=1024, and 76 % [56.57, 88.50] (19/25 cells) across all five capture depths at b=512, with the same cells failing at every layer in each budget. Neither the 12-pp Qwen-vs-Llama gap at b=1024 nor the 20-pp gap at b=512 is a capture-layer tuning miss; both reflect architectural differences in Llama's attention patterns or selection-step bottlenecks. The retrieval encoder picks the same residency set regardless of which capture layer feeds the eviction scorer. This validates the "ports without modification" claim: capture-layer choice has no measurable effect on Llama at any budget we tested.

The three EVOKE variants (`kv_restore`, `attention`, `recovery_aware`) converge to identical NIAH pass rates (76/88/88 %) confirming the recovery-aware fix is transparent on single-needle retrieval. Agent_bench on Llama 3.1 8B: EVOKE strategies (breadcrumb, kv_restore, attention) and InfLLM all PASS the planted-config probe at every budget; recency, streaming_llm, evoke_discard, h2o all fail (the same divide as Qwen).

A.10 Concurrency and PCIe bandwidth

We drive $N \in \{1, 4, 8, 16\}$ concurrent sessions through the same evoke_serve instance (Qwen 2.5 7B, b=1024, smart-recovery on, 14-turn planted-fact workload per session). Each level repeats for five rounds so the per-turn latency distribution at N=16 is built from 1120 turn samples.

N	probe_ok	n_turn	wall (s)	p50	p95	p99	p999	max
1	5/5	70	38.9	2.71	3.04	3.10	3.10	3.19
4	20/20	280	51.5	3.56	4.52	4.87	4.87	4.90
8	40/40	560	98.8	6.89	8.95	9.10	9.14	9.16
16	80/80	1120	205.0	14.86	17.77	18.01	18.13	18.14

Table 10: Per-turn latency in seconds at varying concurrency. Probe pass is 100 % at every N; per-round wall-clock at N=16 varies by under 0.3 % across the five rounds.

Two findings the single-shot runs did not surface. The per-turn latency distribution is tightly banded under sustained load: at every N, the p999 latency is within 0.13 s of the p99 latency. There is no long tail; the latency band is set by the GPU's per-decode compute time. The 100 % probe-OK rate holds across all 80 N=16 sessions, so the K/V splice primitive holds up to 16 concurrent evicting sessions without correctness loss.

Per-session wall-clock scales roughly linearly with N ($\sim$23$\times$ at 16$\times$ concurrency). The dominant scaling factor is GPU compute contention (a single 16 GB GPU runs one Qwen 2.5 7B forward pass at a time), not the PCIe save/load path. Back-of-envelope: at 56 KiB per token times 128-token blocks ($\sim$7 MiB per block), N simultaneous eviction events of k blocks each move $7Nk$ MiB across PCIe Gen4 16 GB/s, $\sim$0.4 ms per block per session; at N=16 and a 4-block eviction burst the aggregate PCIe traffic is 448 MiB at $\sim$28 ms serial worst case. The PCIe pessimism is not the binding constraint here; per-GPU compute throughput is. Production multi-tenant scaling (the next-paper concern, Section 9) would need tensor-parallel or paged-virtual KV layout to lift the compute ceiling, which this paper does not claim to provide.

B Reproducing the Numbers

All numbers in Section 7 were produced on a single GPU host (RTX 4070 Ti SUPER, 16 GB VRAM, CUDA 13.1, Python 3.12) against Qwen 2.5 7B Instruct (`Qwen2.5-7B-Instruct-Q4_K_M.gguf` from the official Qwen GGUF release). The EVOKE policy layer runs in Python; the C primitives live in the forked llama.cpp.

B.1 Build the fork

```
# Clone the EVOKE-modified llama.cpp fork
git clone https://github.com/Anyesh/llama.cpp.git llama.cpp
cd llama.cpp

# Build with CUDA + shared library output
```

```
cmake -B build -DLLAMA_CUDA=ON -DBUILD_SHARED_LIBS=ON
cmake --build build --config Release
# Output: build/bin/libllama.so
```

B.2 Install the EVOKE Python package

```
git clone https://github.com/Anyesh/EVOKE.git evoke
cd evoke
uv sync --extra server

# Point the engine binding at the EVOKE fork build
export LLAMA_CPP_LIB=/abs/path/to/llama.cpp/build/bin/libllama.so
export EVOKE_MODEL_PATH=/abs/path/to/Qwen2.5-7B-Instruct-Q4_K_M.gguf
```

B.3 Section 7.1 latency table (`profile_recover.py`)

```
uv run python scripts/profile_recover.py
```

The script does five warmup cycles at 40 tokens to settle CUDA graph compilation, then sweeps block sizes 20, 40, 80, 160, 320, 640, 1280 with five trials each, reports the mean save time, mean load time, and mean re-prefill time. The numbers in Section 7.1 are the means.

B.4 Appendix A.2 eviction demo (`eviction_demo.py`)

```
# In one terminal: start the server
LLAMA_CPP_LIB=$LLAMA_CPP_LIB EVOKE_MODEL_PATH=$EVOKE_MODEL_PATH \
EVOKE_HOST=0.0.0.0 EVOKE_BUDGET=1024 EVOKE_MODEL_NAME=qwen25 \
uv run python scripts/evoke_serve.py

# In another terminal: drive the demo
EVOKE_SERVER='http://localhost:8000' EVOKE_MODEL_NAME='qwen25' \
  uv run python scripts/eviction_demo.py
```

For the Qwen 3.5 hybrid run, add `EVOKE_SUPPRESS_THINKING_STRIP=1` to the server env and load `Qwen3.5-9B-Q4_K_M.gguf`.

B.5 Section 7.6 head-to-head baselines (`baseline_bench.py`)

```
EVOKE_SERVER='http://eval-host:8000' \
EVOKE_SSH_HOST='user@eval-host' \
EVOKE_LIB_PATH='/path/to/libllama.so/on/eval/host' \
EVOKE_MODEL_PATH='/path/to/Qwen2.5-7B-Instruct-Q4_K_M.gguf' \
EVOKE_BUDGET=1024 EVOKE_N_CTX=16384 EVOKE_MODEL_NAME=qwen25 \
  uv run python scripts/baseline_bench.py
```

The bench drives three policies (`no_eviction`, `truncate`, `evoke`) over the same 14-turn conversation via SSH-launched server instances. The inter-policy launch wrapper is in `scripts/` (commit `2b3cbea`).

B.6 Section 7.3 and appendix A.1 agentic eval (`agent_bench.py`)

```
EVOKE_MODEL_PATH=$EVOKE_MODEL_PATH \
EVOKE_BUDGETS=512,1024,2048 \
  uv run python scripts/agent_bench.py
```

The script runs five strategies (`recency`, `streaming_llm`, `evoke_discard`, `evoke_breadcrumb`, `evoke_kv_restore`) at each budget. For the Appendix A.1 attention row, set `EVOKE_W_ATTENTION=0.5` and `EVOKE_ATTN_LAYER=20`, which enables the `evoke_attention` strategy.

B.7 Appendix A.6 keepalive workload (`agent_bench_keepalive.py`)

```
EVOKE_MODEL_PATH=$EVOKE_MODEL_PATH \
EVOKE_BUDGETS=512,1024,2048 \
  uv run python scripts/agent_bench_keepalive.py
```

The keepalive variant has each filler-file content reference the central `config.py` so the model's attention keeps coming back to the config block, which is what the attention scorer exploits.

B.8 Multi-session pool sanity check (`verify_multi_session.py`)

```
# Server must already be running (see A.4)
EVOKE_SERVER='http://localhost:8000' EVOKE_MODEL_NAME='qwen25' \
  uv run python scripts/verify_multi_session.py
```

The script drives two sessions with distinct planted codewords through interleaved chat requests and asserts that each session retrieves its own codeword, confirming state-swap correctness.

C The Fork

The llama.cpp fork is hosted at `github.com/Anyesh/llama.cpp`. The fork tracks upstream `ggml-org/llama.cpp` and adds the EVOKE-specific commits listed below.

C.1 EVOKE-specific commits in the fork

Commit	Title	What it adds
`1f37e75e7`	EVOKE: iSWA dual-cache support in `kv_block_save/load` (#41)	`llama_get_kv_cache_swa()` and an extended save/load buffer layout that covers both base and SWA sub-caches on Gemma 3 and 4
`151f1cf93`	EVOKE: multi-layer attention capture (up to 16 layers per decode)	`llama_attn_capture_set_layers(ctx, layers, n)` writes one slab per layer in a single decode
`713f202e8`	EVOKE: attention-weight capture primitive	The original single-layer side-compute path: `llama_attn_capture_set_layer`, `_set_buffer`, `_get_dims`, `_get_written`
`9b6232446`	EVOKE: attention-only `seq_rm` and `seq_add` primitives	`llama_kv_block_seq_rm` and `llama_kv_block_seq_add` reach the attention sub-cache directly via `llama_get_kv_cache`, for "no-shift" eviction modes
`a29599f4c`	EVOKE: `kv_block` primitives now cover hybrid memory and mrope models	Extends `llama_get_kv_cache` to unwrap `llama_memory_hybrid` via `get_mem_attn()` and `llama_memory_hybrid_iswa` via `get_mem_attn()->get_base()`. Removes the mrope `seq_add()` and `get_can_shift()` guards.

C.2 The two recovery primitives, verbatim

From `include/llama.h`:

```
LLAMA_API size_t llama_kv_block_save(
    struct llama_context * ctx,
            llama_seq_id   seq_id,
               llama_pos   p0,
               llama_pos   p1,
                 uint8_t * dst,
                  size_t   cap);

// Load a buffer produced by llama_kv_block_save into seq_id starting at new_p0.
LLAMA_API bool llama_kv_block_load(
    struct llama_context * ctx,
             llama_seq_id   seq_id,
           const uint8_t * src,
                  size_t   size,
               llama_pos   new_p0);
```

`save` returns the number of bytes written (or zero on error, e.g. `cap` too small). `load` returns true on success. The caller is responsible for the buffer allocation in both directions.

C.3 Build instructions

The fork builds against the upstream llama.cpp toolchain. Requirements: CMake $\geq$ 3.14, a C++17 compiler, and (for the GPU path) CUDA Toolkit 11.8 or later (we tested with 13.1).

```
cmake -B build -DLLAMA_CUDA=ON -DBUILD_SHARED_LIBS=ON
cmake --build build --config Release
```

The Python binding (`evoke/_engine_lib.py` and `evoke/llama_engine.py`) loads the shared library indicated by the `LLAMA_CPP_LIB` environment variable (a file path). It then derives `LLAMA_CPP_LIB_PATH` (a directory) for `llama-cpp-python` 0.3.x compatibility so a single loaded module backs both the Python wrapper and the EVOKE ctypes binding. Mixing two builds (the upstream wheel and the fork) causes layout-mismatch crashes; using a single fork build everywhere is the supported configuration.

C.4 Attention-capture C API surface, verbatim

The full C API for attention-weight capture, summarized in Section 3.3, added to `include/llama.h`:

```
LLAMA_API void   llama_attn_capture_set_layer (llama_context *, int32_t layer);
LLAMA_API void   llama_attn_capture_set_layers(llama_context *, const int32_t *
    layers, int32_t n);
LLAMA_API void   llama_attn_capture_set_buffer(llama_context *, float * dst,
    size_t cap_floats);
LLAMA_API void   llama_attn_capture_get_dims  (const llama_context *, int32_t *
    n_layers,
                                               int32_t * n_q, int32_t * n_h,
    int32_t * n_kv);
LLAMA_API size_t llama_attn_capture_get_written(const llama_context *);
```